

Login (Front-end)

#Passou CT_1: E-mail e Senha Corretos com Usuário Existente

Dado: que o usuário possui cadastro ativo no sistema

Quando: o usuário acessa a tela de login, preenche o campo "E-mail do usuário" com `carlos.silva@example.com` e o campo "Senha do usuário" com `senha123`

E: clica no botão "Entrar"

Então: o sistema autentica o usuário e redireciona para a tela inicial (dashboard)

#Passou CT_2: E-mail com Formato Inválido e Senha Correta

Dado: que o usuário está na tela de login

Quando: o usuário preenche o campo "E-mail do usuário" com `carlos.silva` (sem @) e o campo "Senha do usuário" com `senha123`

E: clica no botão "Entrar"

Então: o sistema exibe mensagem de erro de autenticação e não permite o acesso

#Passou CT_3: E-mail Correto e Senha Incorreta

Dado: que o usuário está na tela de login

Quando: o usuário preenche o campo "E-mail do usuário" com `carlos.silva@example.com` e o campo "Senha do usuário" com `senhaerrada`

E: clica no botão "Entrar"

Então: o sistema exibe mensagem de erro de autenticação e não permite o acesso

#Passou CT_4: E-mail e Senha Ambos em Formato Incorreto

Dado: que o usuário está na tela de login

Quando: o usuário preenche o campo "E-mail do usuário" com `usuarioinvalido` e o campo "Senha do usuário" com `123`

E: clica no botão "Entrar"

Então: o sistema exibe mensagem de erro de autenticação e não permite o acesso

#Passou CT_5: Login com Campo de E-mail Vazio

Dado: que o usuário está na tela de login

Quando: o usuário deixa o campo "E-mail do usuário" em branco e preenche a senha com `senha123`

E: clica no botão "Entrar"

Então: o sistema exibe mensagem de validação indicando que o campo e-mail é obrigatório e não prossegue com o login

#Passou CT_6: Login com Campo de Senha Vazio

Dado: que o usuário está na tela de login

Quando: o usuário preenche o campo "E-mail do usuário" com `carlos.silva@example.com` e deixa o campo "Senha do usuário" em branco

E: clica no botão "Entrar"

Então: o sistema exibe mensagem de validação indicando que o campo senha é obrigatório e não prossegue com o login

Menu (Exploratórios — Frontend)#

#Passou CT_7: Recolher Menu ao Clicar no Botão

Dado: que o usuário está autenticado e o menu lateral está expandido

Quando: o usuário clica no botão de recolher menu

Então: os ícones do menu são recolhidos de forma responsiva, sem quebrar o layout



(Responsivos sem quebrar o layout)

#Passou CT_8: Ícones do Menu São Clicáveis e Navegam para as Telas Corretas

Dado: que o usuário está autenticado e o menu está visível

Quando: o usuário clica em cada ícone disponível no menu

Então: o sistema redireciona para a respectiva tela de cada ícone sem erros

#Passou CT_9: Clicar no Ícone de Sair Desloga o Usuário

Dado: que o usuário está autenticado e na aplicação

Quando: o usuário clica no ícone "Sair" no menu

Então: a sessão é encerrada e o usuário é redirecionado para a tela de login

#Passou CT_10: Clicar nos Ícones FADETEC, SIFSOFT e IFNMG Retorna à Tela Inicial

Dado: que o usuário está autenticado em qualquer tela do sistema

Quando: o usuário clica nos ícones FADETEC, SIFSOFT ou IFNMG

Então: o sistema retorna para a tela inicial (dashboard) da aplicação

#Passou CT_11: Item Clicado no Menu Fica em Destaque

Dado: que o usuário está autenticado e navegando pelo menu

Quando: o usuário clica em um item do menu

Então: o item clicado fica visivelmente em destaque (ex: cor diferente, borda ativa), indicando a seleção ativa

#Passou CT_12: Item "Usuário" Exibe Nome e E-mail do Usuário Logado

Dado: que o usuário está autenticado com nome e e-mail longos (ex: "Carlos Alberto de Souza Silva Pereira" / "carlos.alberto.souza.silva.pereira@instituicao.edu.br")

Quando: o usuário visualiza o item "Usuário" no menu

Então: o sistema exibe o nome e o e-mail completos, sem truncar de forma incorreta ou quebrar o layout do menu

Contas Bancárias — Cadastro (API — POST /bankaccount)

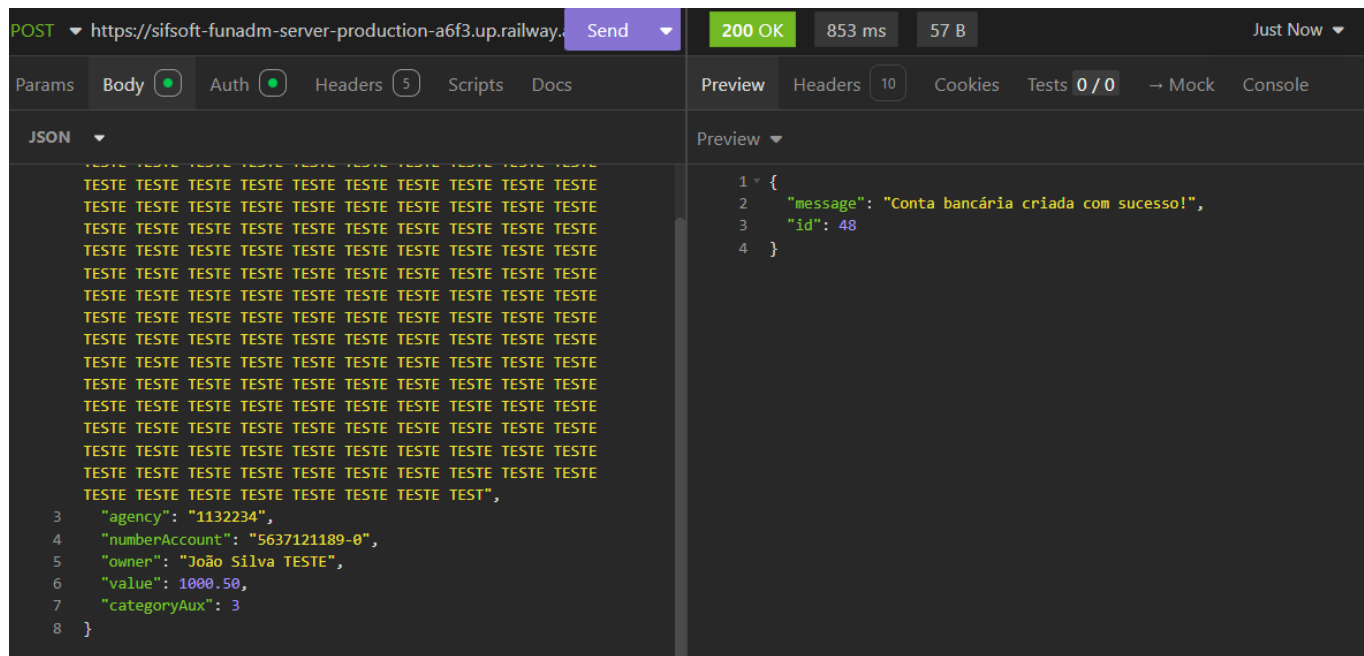
#Passou CT_13: Cadastro com Todos os Campos Válidos

Dado: que o usuário está autenticado com token válido

Quando: é enviada uma requisição POST para `/bankaccount` com o body:

```
{
  "name": "Banco do Brasil",
  "agency": "1234",
  "numberAccount": "56789-0",
  "owner": "João Silva",
  "value": "1000.50",
  "categoryAux": 1
}
```

Então: o sistema retorna HTTP 200 com a mensagem "Inserido com sucesso!" e o `id` da conta criada



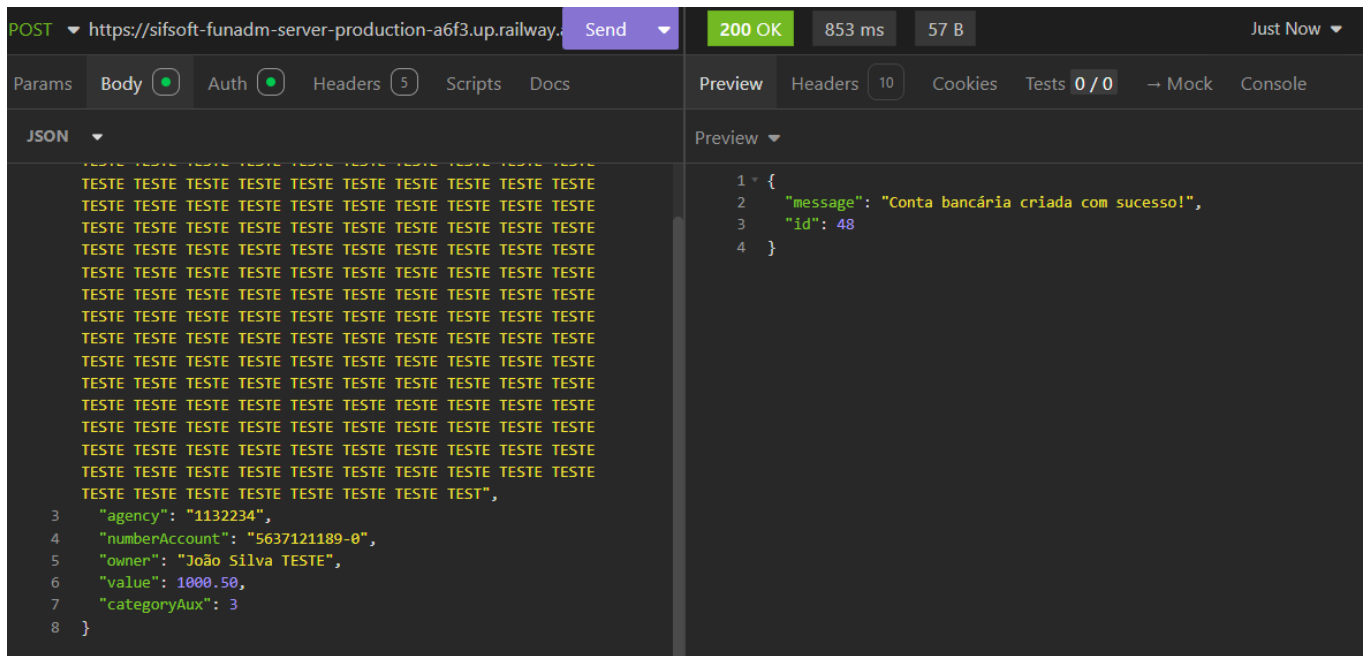
#Passou CT_14: Cadastro com Nome do Banco com 1000 Caracteres (Limite Máximo)

Dado: que o usuário está autenticado

Quando: é enviada uma requisição POST para `/bankaccount` com o campo `name` contendo exatamente 1000 caracteres (string "A" repetida 1000 vezes)

E: os demais campos estão preenchidos com valores válidos

Então: o sistema retorna HTTP 200 com "Inserido com sucesso!"



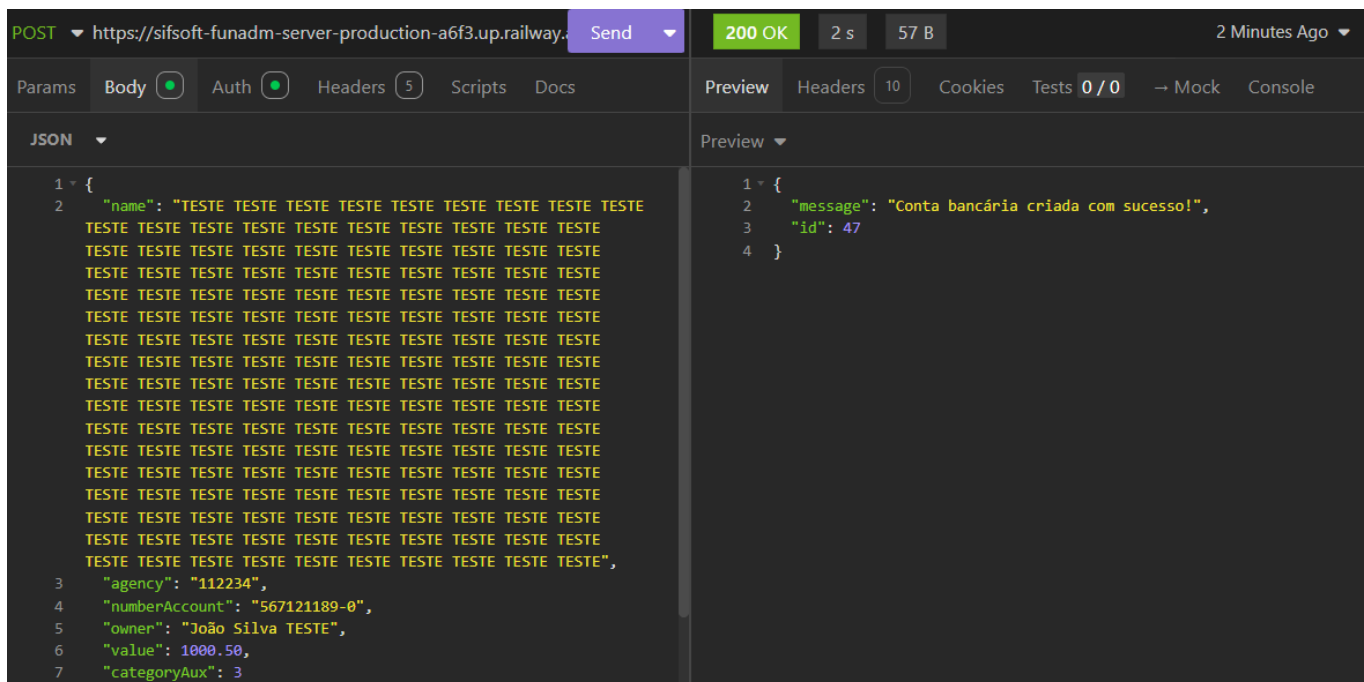
#Falhou CT_15: Cadastro com Nome do Banco com 1001 Caracteres (Acima do Limite)

Dado: que o usuário está autenticado

Quando: é enviada uma requisição POST para `/bankaccount` com o campo `name` contendo 1001 caracteres

E: os demais campos estão preenchidos com valores válidos

Então: o sistema retorna HTTP 500 — Internal Server Error ou rejeita a entrada com erro de validação



☐ (Aceitou a requisição, não está sendo feito o controle de caracteres)

#Sugestão #Passou **CT_16: Cadastro com Nome do Banco Nulo (Campo Obrigatório)**

Dado: que o usuário está autenticado

Quando: é enviada uma requisição POST para `/bankaccount` com o body:

```
{
  "name": null,
  "agency": "1234",
  "numberAccount": "156178110",
  "owner": "Fadetec",
  "value": 1000,
  "categoryAux": 3
}
```

Então: o sistema retorna HTTP 500 — Internal Server Error ou mensagem de campo obrigatório

POST https://sifsoft-funadm-server-production-a6f3.up.railway. 500 Internal Server Error 352 ms 354 B Just Now

Params Body Auth Headers 5 Scripts Docs Preview Headers 11 Cookies Tests 0/0 → Mock Console

JSON

```
1 {
2   "name": null,
3   "agency": "113223334",
4   "numberAccount": "56371321189-0",
5   "owner": "João Silva TESTE",
6   "value": 1000.50,
7   "categoryAux": 3
8 }
```

Preview

```
1 {
2   "message": "Erro ao criar a Conta bancária!",
3   "error": "\nInvalid `prisma.bankAccount.create()`\n\ninvocation:\n{\n  data: {\n    agency: \"113223334\",\n    numberAccount: \"56371321189-0\",\n    owner: \"João Silva TESTE\",\n    value: new Prisma.Decimal(\"1000.5\"),\n    categoryAux: 3,\n    name: String\n  }\n}\n\nArgument `name` must not be null."
4 }
```

#Falhou **CT_17: Cadastro com Nome do Banco Vazio**

Dado: que o usuário está autenticado

Quando: é enviada uma requisição POST para `/bankaccount` com o campo `name` como string vazia `""`

Então: o sistema retorna erro indicando que o campo não pode ser vazio

☐ (A API aceitou o campo vazio, mesmo sendo um campo obrigatório para preenchimento)

POST https://sifsoft-funadm-server-production-a6f3.up.railway.app/ Send 200 OK 365 ms 57 B Just Now

Params Body Auth Headers 5 Scripts Docs Preview Headers 10 Cookies Tests 0/0 → Mock Console

JSON

```
1 {
2   "name": "",
3   "agency": "113223334",
4   "numberAccount": "56371321189-0",
5   "owner": "João Silva TESTE",
6   "value": 1000.50,
7   "categoryAux": 3
8 }
```

Preview

```
1 {
2   "message": "Conta bancária criada com sucesso!",
3   "id": 49
4 }
```

#Passou CT_18: Cadastro com Nome do Banco Contendo Apenas Caracteres Especiais

Dado: que o usuário está autenticado

Quando: é enviada uma requisição POST para /bankaccount com o body:

```
{
  "name": "@#$$%`&*()",
  "agency": "1234",
  "numberAccount": "56711",
  "owner": "João Silva",
  "value": "1000.50",
  "categoryAux": 1
}
```

Então: o sistema retorna HTTP 200

POST https://cash-management-teste.up.railway.app/bankacco Send 200 OK 909 ms 159 B Just Now

Params Body Auth Headers 4 Scripts Docs Preview Headers 9 Cookies Tests 0/0 → Mock Console

JSON

```
1 {
2   "name": "@#$$%`&*()",
3   "agency": "1234",
4   "numberAccount": "56711",
5   "owner": "Fadetec",
6   "value": 1000.50,
7   "categoryAux": 3
8 }
```

Preview

```
1 {
2   "id": 122,
3   "name": "@#$$%`&*()",
4   "agency": "1234",
5   "numberAccount": "56711",
6   "owner": "Fadetec",
7   "value": 1000.5,
8   "categoryAux": 3,
9   "created_at": "2026-06-17T16:56:48.979Z"
10 }
```

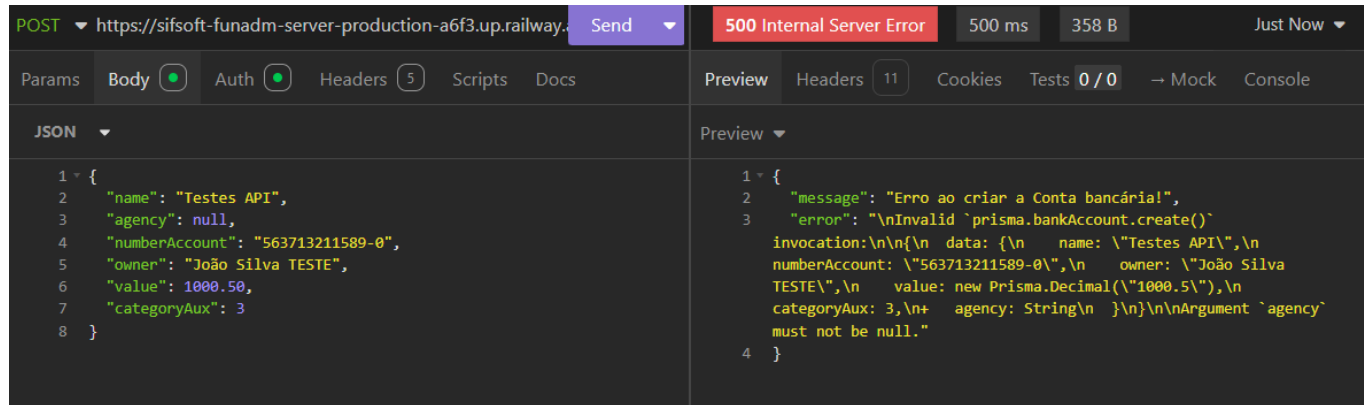
#Sugestão #Passou CT_19: Cadastro com Agência Nula (Campo Obrigatório)

Dado: que o usuário está autenticado

Quando: é enviada uma requisição POST para `/bankaccount` com o campo `agency` como `null`

E: os demais campos estão preenchidos com valores válidos

Então: o sistema retorna erro indicando campo obrigatório



The screenshot shows a REST client interface with a POST request to `https://sifsoft-funadm-server-production-a6f3.up.railway...`. The request body is a JSON object with the following fields: `name` (Testes API), `agency` (null), `numberAccount` (563713211589-0), `owner` (João Silva TESTE), `value` (1000.50), and `categoryAux` (3). The response is a 500 Internal Server Error with a status of 500 ms and 358 B. The response body is a JSON object with the following fields: `message` (Erro ao criar a Conta bancária!), `error` (Invalid `prisma.bankAccount.create()` invocation), `data` (an object with the same fields as the request), and `value` (new Prisma.Decimal("1000.50")).

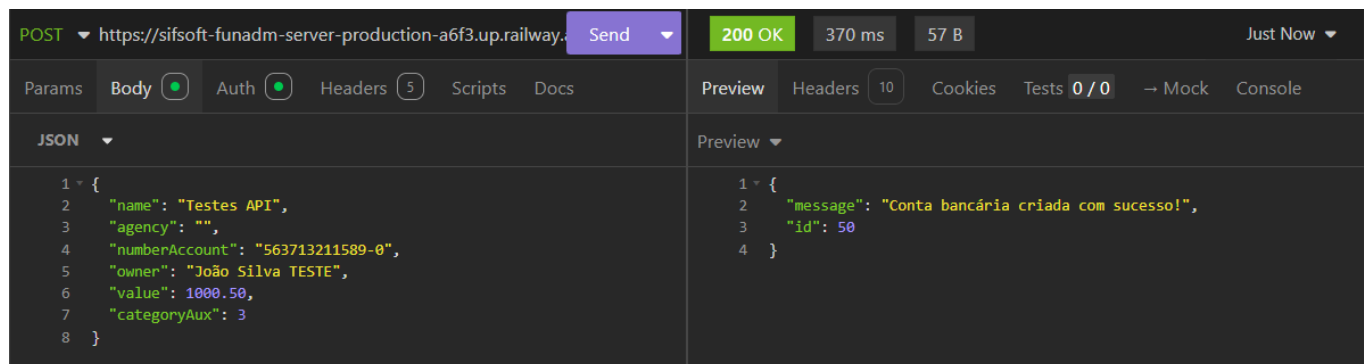
#Falhou CT_20: Cadastro com Agência Vazio

Dado: que o usuário está autenticado

Quando: é enviada uma requisição POST para `/bankaccount` com o campo `agency` como string vazia `""`

Então: o sistema retorna erro indicando que o campo não pode ser vazio

☐ (A API aceitou o campo vazio, mesmo sendo um campo obrigatório para preenchimento)



The screenshot shows a REST client interface with a POST request to `https://sifsoft-funadm-server-production-a6f3.up.railway...`. The request body is a JSON object with the following fields: `name` (Testes API), `agency` (""), `numberAccount` (563713211589-0), `owner` (João Silva TESTE), `value` (1000.50), and `categoryAux` (3). The response is a 200 OK with a status of 370 ms and 57 B. The response body is a JSON object with the following fields: `message` (Conta bancária criada com sucesso!), `id` (50), and `value` (new Prisma.Decimal("1000.50")).

#Falhou CT_21: Cadastro com Saldo Negativo

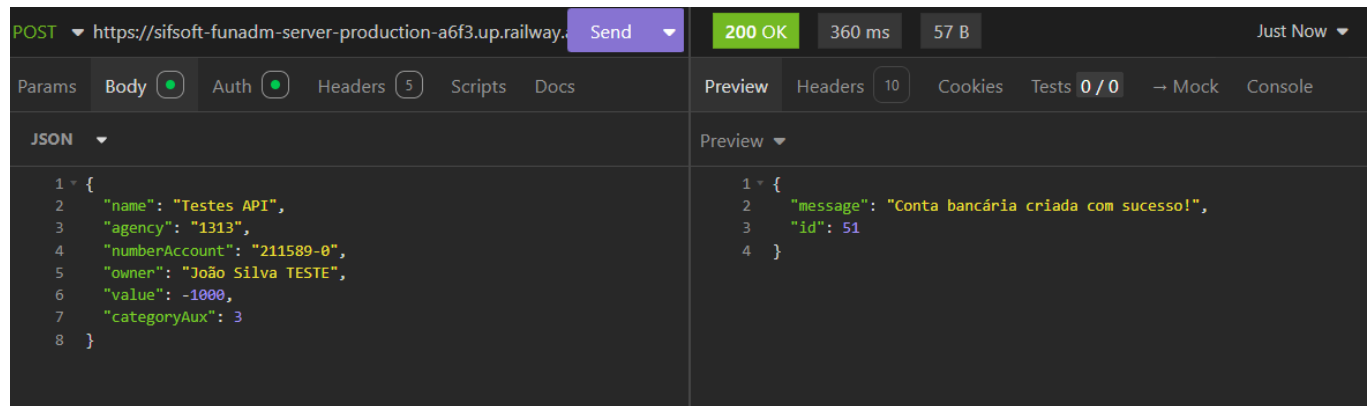
Dado: que o usuário está autenticado e o tipo de conta é diferente de "Projeto" (`categoryAux` != código de Projeto)

Quando: é enviada uma requisição POST para `/bankaccount` com o body:

```
{
  "name": "Banco X",
  "agency": "0001",
  "numberAccount": "00001-0",
  "owner": "Maria",
  "value": "-500.00",
  "categoryAux": 2
}
```

Então: o sistema valida e retorna erro ou aceita, conforme regra de negócio definida

☐ (Não era para o sistema aceitar, pois conforme o front não é deixado digitar caracteres)



#Falhou CT_22: Cadastro com Tipo "Projeto" e Saldo Informado

Dado: que o usuário está autenticado

Quando: é enviada uma requisição POST para `/bankaccount` com `categoryAux` correspondente ao tipo "Projeto" e o campo `value` preenchido com `"100"`

Então: o sistema ignora o saldo ou retorna erro, pois o campo não é habilitado para contas do tipo Projeto

☐ (Não é feito tratamento dessa condição do campo 'Saldo' não ser preenchido quando o tipo é 'Projeto')

POST <https://sifsoft-funadm-server-production-a6f3.up.railway...> Send 200 OK 1.53 s 57 B Just Now

Params Body Auth Headers 5 Scripts Docs Preview Headers 10 Cookies Tests 0/0 → Mock Console

JSON

```
1 {
2   "name": "Testes API",
3   "agency": "1313",
4   "numberAccount": "2111589-0",
5   "owner": "João Silva TESTE",
6   "value": 100,
7   "categoryAux": 3
8 }
```

Preview

```
1 {
2   "message": "Conta bancária criada com sucesso!",
3   "id": 52
4 }
```

#Passou CT_23: Cadastro com Tipo de Conta Nulo (Campo Obrigatório)

Dado: que o usuário está autenticado

Quando: é enviada uma requisição POST para `/bankaccount` com o body:

```
{
  "name": "Banco X",
  "agency": "0001",
  "numberAccount": "00001-0",
  "owner": "Maria",
  "value": "100.00",
  "categoryAux": null
}
```

Então: o sistema retorna HTTP 500 ou mensagem de campo obrigatório

POST <https://sifsoft-funadm-server-production-a6f3.up.railway...> Send 400 Bad Request 500 ms 132 B Just Now

Params Body Auth Headers 5 Scripts Docs Preview Headers 10 Cookies Tests 0/0 → Mock Console

JSON

```
1 {
2   "name": "Testes API",
3   "agency": "1313",
4   "numberAccount": "21311589-0",
5   "owner": "João Silva TESTE",
6   "value": 100,
7   "categoryAux": null
8 }
```

Preview

```
1 {
2   "message": "Categoria inválida! As categorias permitidas são:
  'Projetos', 'Administrativa', 'Fundo de reserva' e
  'Investimento'."
3 }
```

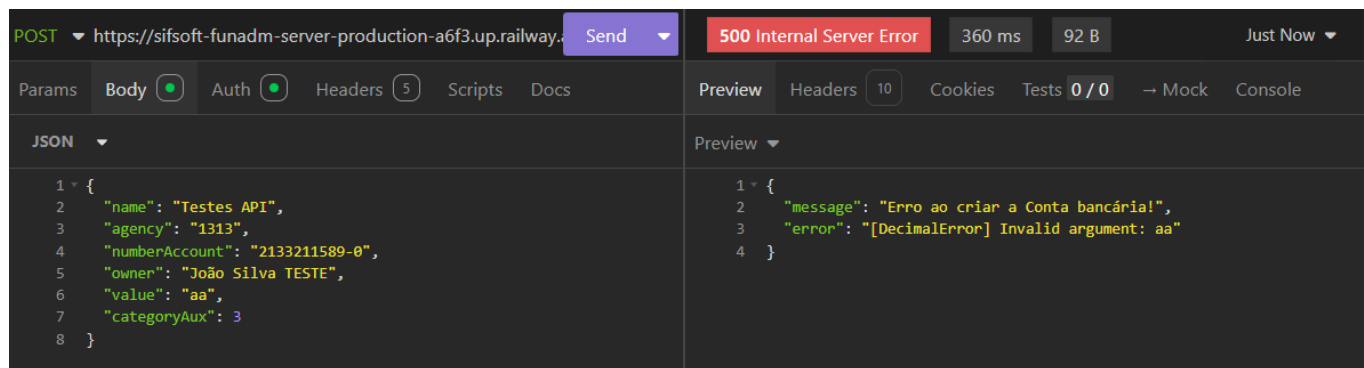
#Passou CT_24: Cadastro com Saldo Contendo Letras no Valor

Dado: que o usuário está autenticado

Quando: é enviada uma requisição POST para `/bankaccount` com `value = "abc"`

E: os demais campos estão preenchidos com valores válidos

Então: o sistema retorna erro de tipo inválido ou HTTP 500



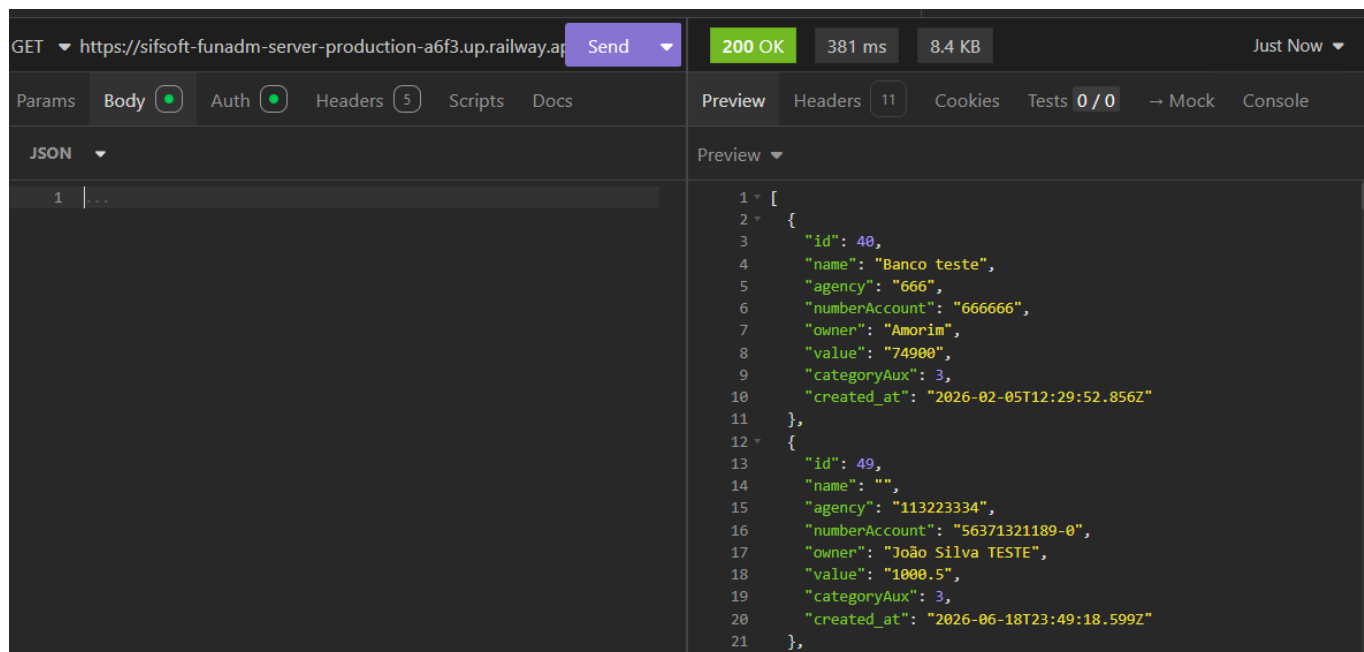
Contas Bancárias — Listagem (API — GET /bankaccount)

#Passou CT_25: Listar Todas as Contas Bancárias com Token Válido

Dado: que o usuário está autenticado com token válido

Quando: é enviada uma requisição GET para /bankaccount

Então: o sistema retorna HTTP 200 com a lista de contas bancárias cadastradas

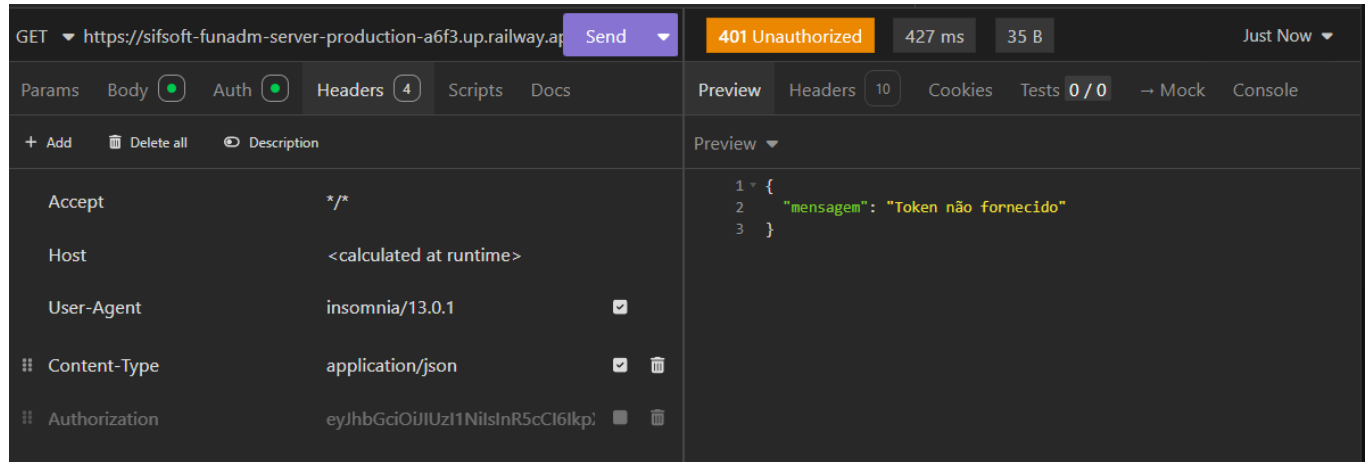


#Passou CT_26: Listar Contas Bancárias sem Token

Dado: que o usuário não está autenticado

Quando: é enviada uma requisição GET para `/bankaccount` sem o header `Authorization`

Então: o sistema retorna HTTP 401 — Unauthorized

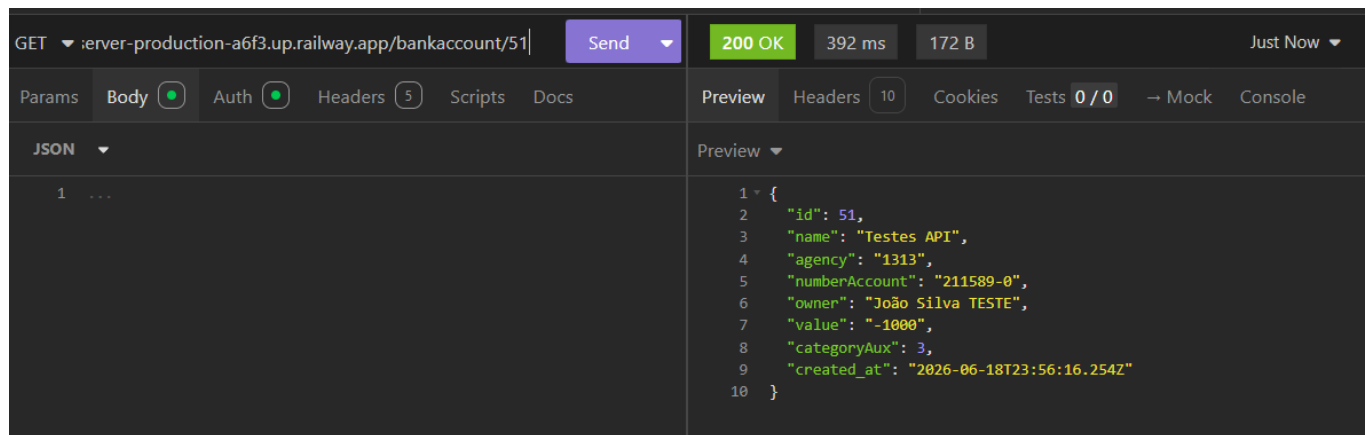


#Passou CT_27: Buscar Conta Bancária por ID Existente (GET `/bankaccount/{id}`)

Dado: que existe uma conta bancária com `id = 51` no sistema e o usuário está autenticado

Quando: é enviada uma requisição GET para `/bankaccount/51`

Então: o sistema retorna HTTP 200 com os dados da conta correspondente

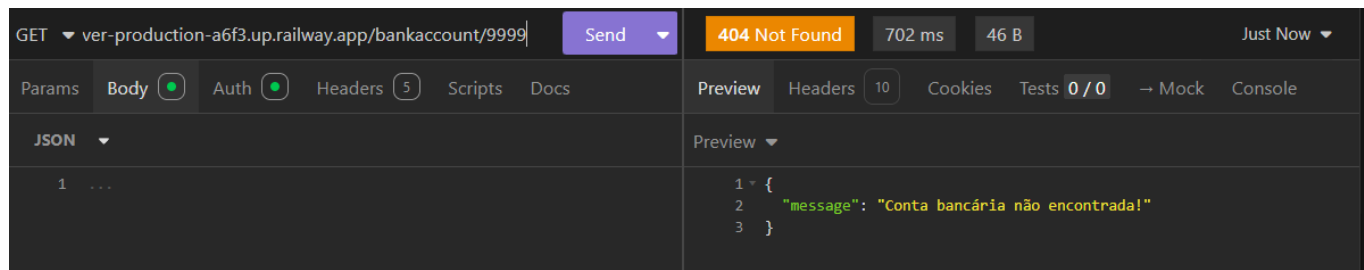


#Passou CT_28: Buscar Conta Bancária por ID Inexistente

Dado: que o usuário está autenticado

Quando: é enviada uma requisição GET para `/bankaccount/9999` (ID não cadastrado)

Então: o sistema retorna HTTP 404 — "Conta bancária não encontrada!"



Contas Bancárias — Edição (API — PUT /bankaccount/{id})

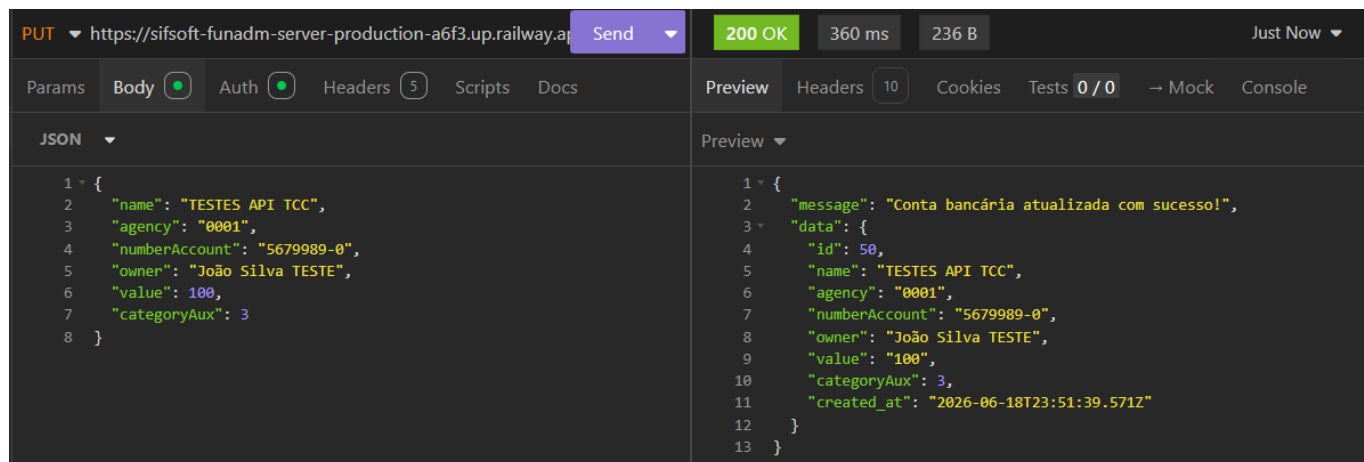
#Passou CT_29: Editar Nome do Banco com Valor Válido

Dado: que existe uma conta bancária com `id = 50` e o usuário está autenticado

Quando: é enviada uma requisição PUT para `/bankaccount/50` com o body:

```
{
  "name": "TESTES API TCC",
  "agency": "0001",
  "numberAccount": "5679989-0",
  "owner": "João Silva TESTE",
  "value": 100,
  "categoryAux": 2
}
```

Então: o sistema retorna HTTP 200 e os dados da conta são atualizados



#Passou CT_30: Tentar Editar Número da Conta

Dado: que existe uma conta bancária com `id = 50` e o usuário está autenticado

Quando: é enviada uma requisição PUT para `/bankaccount/50` com o campo `numberAccount` alterado para `"99999-9"`

Então: o sistema retorna HTTP 200 e os dados da conta são atualizados

The screenshot shows a REST client interface with the following details:

- Method:** PUT
- URL:** `ever-production-a6f3.up.railway.app/bankaccount/50`
- Status:** 200 OK
- Time:** 635 ms
- Size:** 234 B
- Body (JSON):**

```
1 {
2   "name": "TESTES API TCC",
3   "agency": "0001",
4   "numberAccount": "99999-9",
5   "owner": "João Silva TESTE",
6   "value": 100,
7   "categoryAux": 3
8 }
```
- Preview (JSON):**

```
1 {
2   "message": "Conta bancária atualizada com sucesso!",
3   "data": {
4     "id": 50,
5     "name": "TESTES API TCC",
6     "agency": "0001",
7     "numberAccount": "99999-9",
8     "owner": "João Silva TESTE",
9     "value": 100,
10    "categoryAux": 3,
11    "created_at": "2026-06-18T23:51:39.571Z"
12  }
13 }
```

#Passou CT_31: Editar Conta com ID Inexistente

Dado: que o usuário está autenticado

Quando: é enviada uma requisição PUT para `/bankaccount/9999` com body válido

Então: o sistema retorna HTTP 500 — Internal Server Error ou 404

The screenshot shows a REST client interface with the following details:

- Method:** PUT
- URL:** `ever-production-a6f3.up.railway.app/bankaccount/9999`
- Status:** 404 Not Found
- Time:** 362 ms
- Size:** 46 B
- Body (JSON):**

```
1 {
2   "name": "TESTES API TCC",
3   "agency": "0001",
4   "numberAccount": "9911999-9",
5   "owner": "João Silva TESTE",
6   "value": 100,
7   "categoryAux": 3
8 }
```
- Preview (JSON):**

```
1 {
2   "message": "Conta bancária não encontrada."
3 }
```

Contas Bancárias — Exclusão (API — DELETE `/bankaccount/{id}`)

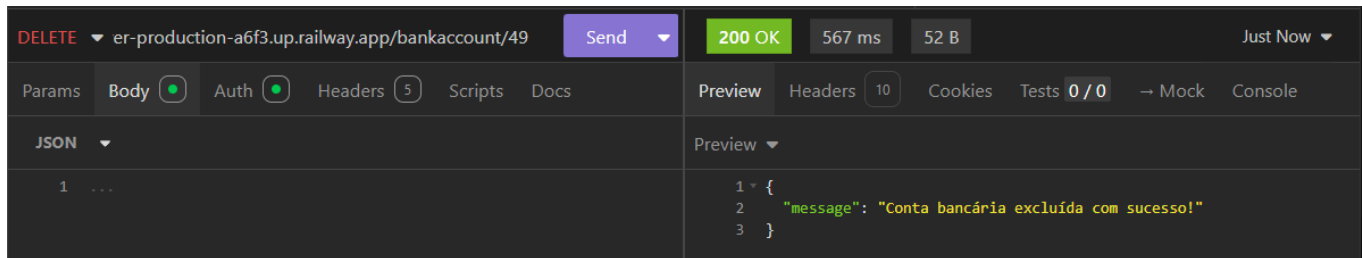
#Passou CT_32: Excluir Conta Bancária sem Vínculo

Dado: que existe uma conta bancária com `id = 5` sem vínculo com projetos, investimentos ou despesas

E: o usuário está autenticado

Quando: é enviada uma requisição DELETE para `/bankaccount/5`

Então: o sistema retorna HTTP 200 — `"Conta bancária excluída com sucesso!"`



#Passou CT_33: Tentar Excluir Conta Bancária Vinculada a um Projeto

Dado: que existe uma conta bancária com `id = 32` vinculada a um projeto ativo

E: o usuário está autenticado

Quando: é enviada uma requisição DELETE para `/bankaccount/32`

Então: o sistema retorna HTTP 500 com mensagem de erro indicando que a conta está em uso

Nº: *

21515161

Projeto: *

Teste

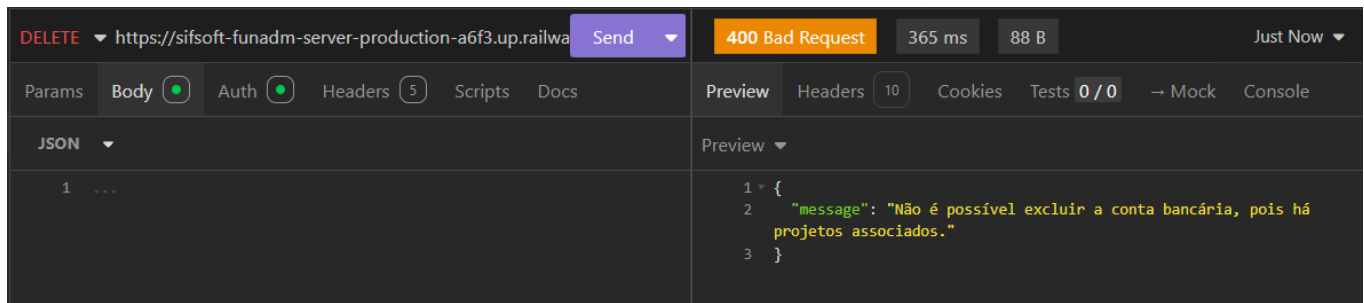
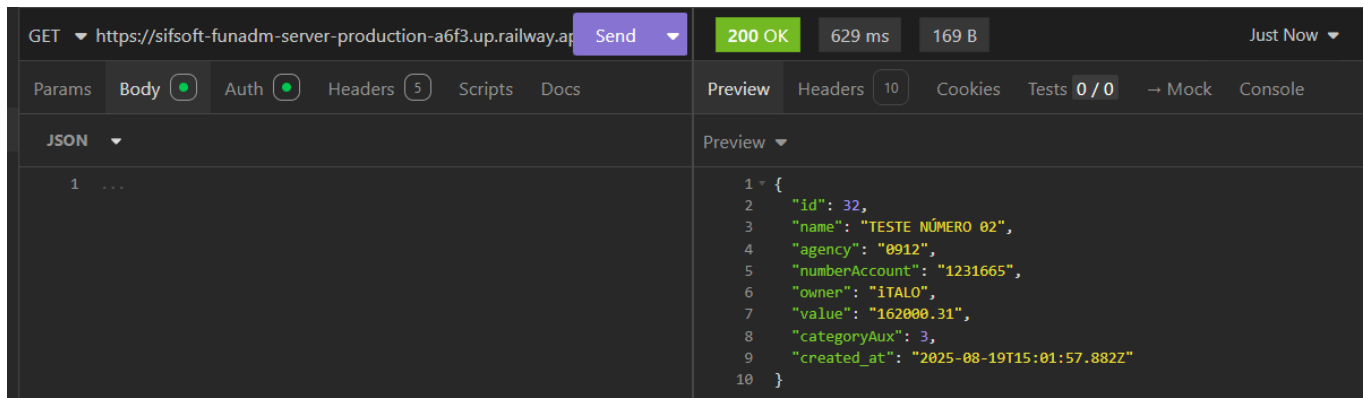
Tipo no plano de trabalho: *

Tipo A



Conta bancária: *

Nome: TESTE NÚMERO 02 - Agência: 0912 - Nº: 1231665

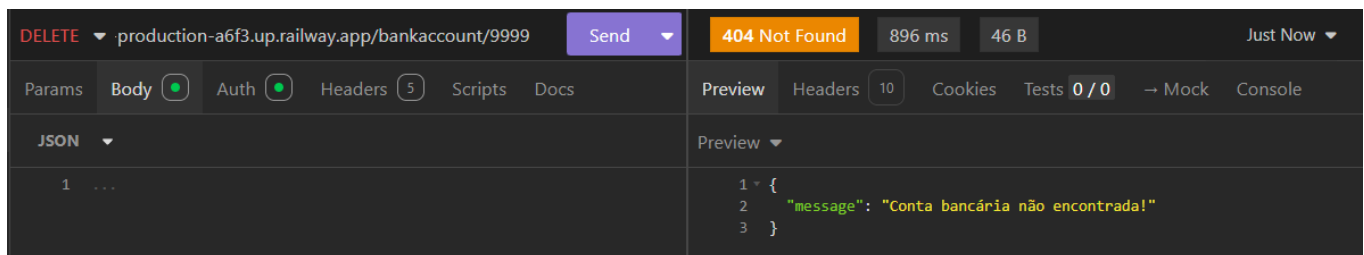


#Passou CT_34: Excluir Conta Bancária com ID Inexistente

Dado: que o usuário está autenticado

Quando: é enviada uma requisição DELETE para /bankaccount/9999

Então: o sistema retorna HTTP 404 — "Conta bancária não encontrada!"



Contas Bancárias — Tela (Frontend)

#Passou CT_35: Visualizar Detalhes da Conta com Nome Muito Longo

Dado: que existe uma conta bancária com nome de 1000 caracteres cadastrada

Quando: o usuário clica no botão "Visualizar" na listagem de contas

Então: a janela flutuante exibe o nome completo sem truncar de forma incorreta, com layout íntegro

Conta Bancária



Nome do banco:

TESTE TESTE TESTE TESTE TESTE TESTE TESTE TESTE TESTE TESTE TESTE TESTE TESTE TESTE TESTE
TESTE TESTE TESTE TESTE TESTE TESTE TESTE TESTE TESTE TESTE TESTE TESTE TESTE TESTE TESTE
TESTE TESTE TESTE TESTE TESTE TESTE TESTE TESTE TESTE TESTE TESTE TESTE TESTE TESTE TESTE
TESTE TESTE TESTE TESTE TESTE TESTE TESTE TESTE TESTE TESTE TESTE TESTE TESTE TESTE TESTE
TESTE TESTE TESTE TESTE TESTE TESTE TESTE TESTE TESTE TESTE TESTE TESTE TESTE TESTE TESTE
TESTE TESTE TESTE TESTE TESTE TESTE TESTE TESTE TESTE TESTE TESTE TESTE TESTE TESTE TESTE
TESTE TESTE TESTE TESTE TESTE TESTE TESTE TESTE TESTE TESTE TESTE TESTE TESTE TESTE TESTE
TESTE TESTE TESTE TESTE TESTE TESTE TESTE TESTE TESTE TESTE TESTE TESTE TESTE TESTE TESTE
TESTE TESTE TESTE TESTE TESTE TESTE TESTE TESTE TESTE TESTE TESTE TESTE TESTE TESTE TESTE
TESTE TESTE TESTE TESTE

Conta bancária:

112234

Número da conta:

567121189-0

Titular:

João Silva TESTE

Saldo:

R\$: 1.000,50

Tipo de conta:

Projeto

#Passou CT_36: Verificar Máscara do Saldo na Visualização

Dado: que existe uma conta bancária com `value = "100"`

Quando: o usuário clica em "Visualizar"

Então: o sistema exibe o saldo formatado como `R$ 100.00`

Conta Bancária

Nome do banco:

Testes API

Conta bancária:

1313

Número da conta:

2111589-0

Titular:

João Silva TESTE

Saldo:

R\$: 100,00

Tipo de conta:

Projeto

The screenshot shows a REST client interface with the following details:

- Method:** GET
- URL:** /server-production-a6f3.up.railway.app/bankaccount/52
- Status:** 200 OK
- Response Time:** 625 ms
- Response Size:** 171 B
- Timestamp:** 1 Minute Ago
- Body:** JSON
- Preview:** Headers (10), Cookies, Tests (0/0), Mock, Console
- JSON Body:**

```
1 {
2   "id": 52,
3   "name": "Testes API",
4   "agency": "1313",
5   "numberAccount": "2111589-0",
6   "owner": "João Silva TESTE",
7   "value": "100",
8   "categoryAux": 3,
9   "created_at": "2026-06-18T23:58:34.703Z"
10 }
```

#Passou CT_37: Botão Editar Redireciona para o Formulário de Edição

Dado: que o usuário está na listagem de contas bancárias

Quando: o usuário clica no botão "Editar" de uma conta

Então: o sistema redireciona imediatamente para o formulário de edição com os dados da conta preenchidos

Nome do banco: *

Testes API

Saldo da conta: *

100,00

Agência: *

1313

Tipo de conta bancária: *

Projetos

Administrativa

Fundo de reserva

Investimento

Número da conta: *

2111589-0

Titular da conta: *

João Silva TESTE

Voltar

Cadastrar

#Passou CT_38: Saldo Não Editável no Formulário de Edição

Dado: que o usuário está no formulário de edição de uma conta bancária

Quando: o usuário tenta editar o campo "Saldo da Conta"

Então: o campo está desabilitado e não permite alteração

Nome do banco: *

Testes API

Saldo da conta: *

100,00

Agência: *

1313

Tipo de conta bancária: *

Projetos

Administrativa

Fundo de reserva

Investimento

Número da conta: *

2111589-0

Titular da conta: *

João Silva TESTE

Voltar

Cadastrar

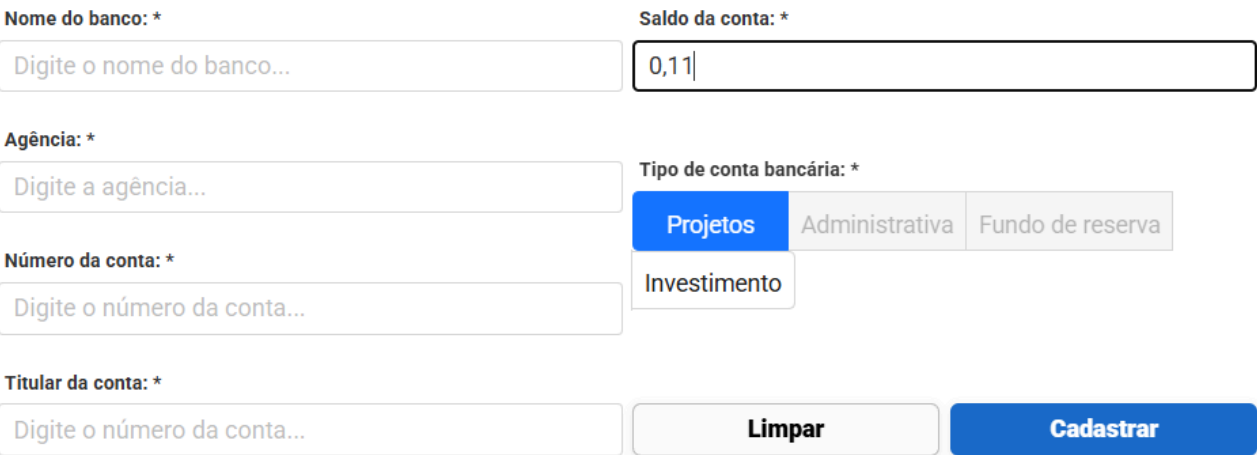
#FalhouGrave #Falhou CT_39: Botão Limpar no Formulário de Cadastro de Conta

Dado: que o usuário preencheu todos os campos do formulário de cadastro de conta bancária

Quando: o usuário clica no botão "Limpar"

Então: todos os campos do formulário são esvaziados/redefinidos ao estado inicial

- ☐ (O botão 'Limpar' apaga todos os campos, porém quando o 'Tipo' está assinalado como 'Investimento' e você clica no botão 'Limpar' é redirecionado para o 'Tipo' 'Projeto' e assim o campo 'Saldo da conta' fica disponível para edição).



Nome do banco: *

Saldo da conta: *

Agência: *

Tipo de conta bancária: *

- Projetos
- Administrativa
- Fundo de reserva
- Investimento

Número da conta: *

Titular da conta: *

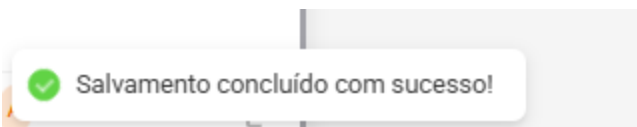
#Sugestão #Passou CT_40: Botão Cadastrar com Campos Obrigatórios Preenchidos

Dado: que o usuário preencheu todos os campos obrigatórios do formulário de conta bancária

Quando: o usuário clica no botão "Cadastrar"

Então: o sistema registra a conta e exibe mensagem de sucesso

- ☐ (A mensagem de sucesso aparece no canto esquerdo abaixo na tela, o que dificulta a visualização)



#Passou CT_41: Campo "Saldo da Conta" Desabilitado ao Selecionar Tipo "Projeto"

Dado: que o usuário está no formulário de cadastro de conta bancária

Quando: o usuário seleciona o tipo "Projeto" no campo "Tipo de Conta Bancária"

Então: o campo "Saldo da Conta" é automaticamente desabilitado na interface

Saldo da conta:

0,00

Tipo de conta bancária:

Projetos. Administrativa. Fundo de reserva. Investimento.

Limpar Cadastr

#Passou CT_42: Preencher Saldo e Depois Mudar Tipo para "Projeto"

Dado: que o usuário preencheu o campo "Saldo da Conta" com um valor válido com tipo 'Investimento'

Quando: o usuário altera o "Tipo de Conta Bancária" para "Projeto"

Então: o campo "Saldo da Conta" é desabilitado e o valor preenchido é limpo

Saldo da conta:

11.111,00

Tipo de conta bancária:

Projetos. Administrativa. Fundo de reserva. Investimento.

Saldo da conta:

0,00

Tipo de conta bancária:

Projetos. Administrativa. Fundo de reserva. Investimento.

Projetos — Cadastro (API — POST /project)

#Passou CT_43: Cadastro de Projeto com Todos os Campos Válidos

Dado: que o usuário está autenticado e existe uma conta bancária, categoria, status e tipo de trabalho cadastrados

Quando: é enviada uma requisição POST para `/project` com o body:

```
{
  "title": "TESTES API - TCC",
  "description": "FOI INSERIDO VIA API - TESTES",
  "value": 10,
  "startDate": "2024-01-01T00:00:00Z",
  "endDate": "2025-01-01T00:00:00Z",
  "instrumentValue": 1,
  "reservefundValue": 1,
  "doaValue": 1,
  "coordinatorName": "TEESTES API",
  "coordinatorEmail": "TESTES.API@gmail.br",
  "bankAccountId": 52,
  "categoryId": 9,
  "statusId": 3,
  "projectTypeWorkId": 4,
  "authorId": 2,
  "projectNumber": 150000000
}
```

Então: o sistema retorna HTTP 200 com "Inserido com sucesso!" e o `id` do projeto criado

The screenshot shows a REST client interface with a POST request to `https://sifsoft-funadm-server-production-a6f3.up.railway...` that returned a `200 OK` status. The request body is a JSON object with the following fields: `title`, `description`, `value`, `startDate`, `endDate`, `instrumentValue`, `reservefundValue`, `doaValue`, `coordinatorName`, `coordinatorEmail`, `bankAccountId`, `categoryId`, `statusId`, `projectTypeWorkId`, `authorId`, and `projectNumber`. The response body is a JSON object with `message` and `id` fields.

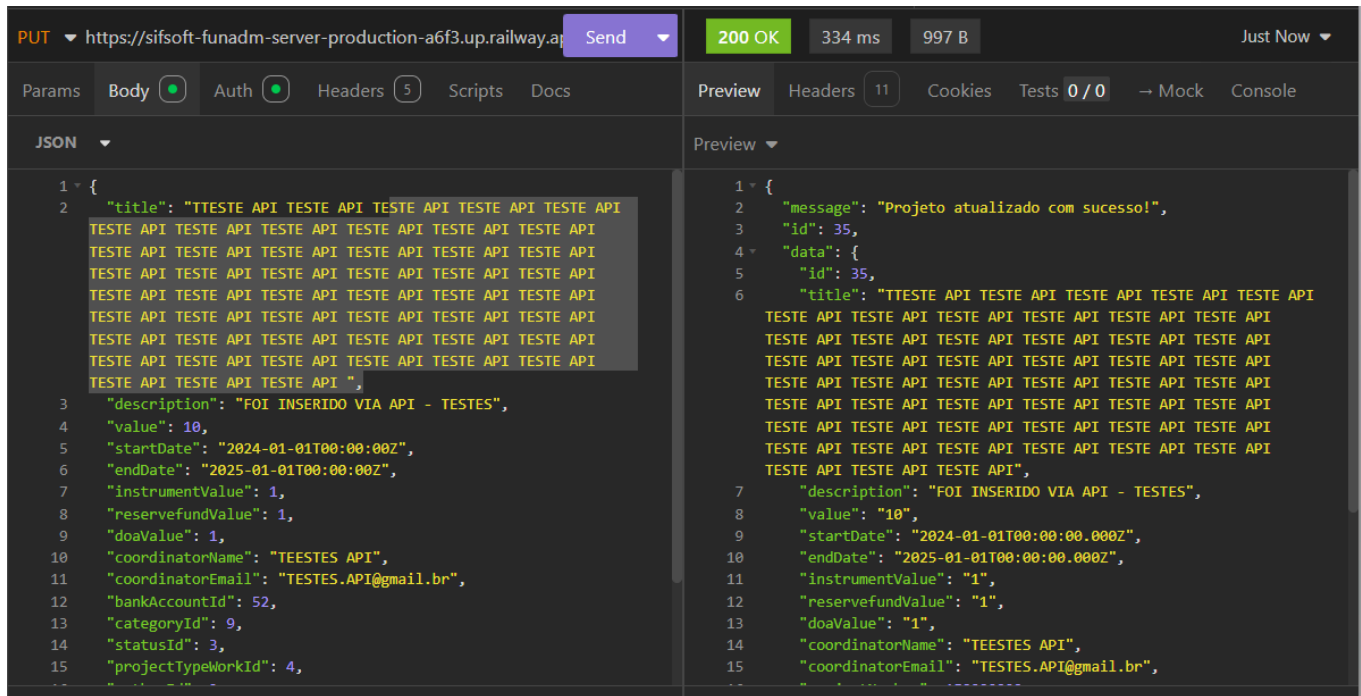
Request	Response
<pre>1 { 2 "title": "TESTES API - TCC", 3 "description": "FOI INSERIDO VIA API - TESTES", 4 "value": 10, 5 "startDate": "2024-01-01T00:00:00Z", 6 "endDate": "2025-01-01T00:00:00Z", 7 "instrumentValue": 1, 8 "reservefundValue": 1, 9 "doaValue": 1, 10 "coordinatorName": "TEESTES API", 11 "coordinatorEmail": "TESTES.API@gmail.br", 12 "bankAccountId": 52, 13 "categoryId": 9, 14 "statusId": 3, 15 "projectTypeWorkId": 4, 16 "authorId": 2, 17 "projectNumber": 150000000 18 }</pre>	<pre>1 { 2 "message": "Projeto criado com sucesso!", 3 "id": 35 4 }</pre>

#Passou CT_44: Cadastro de Projeto com Título de 500 Caracteres (Limite Máximo — char(500))

Dado: que o usuário está autenticado e os campos relacionais estão válidos

Quando: é enviada uma requisição POST para `/project` com o campo `title` contendo exatamente 500 caracteres

Então: o sistema retorna HTTP 200 com "Inserido com sucesso!"



#Sugestão #Passou CT_45: Cadastro de Projeto com Título de 501 Caracteres (Acima do Limite)

Dado: que o usuário está autenticado

Quando: é enviada uma requisição POST para `/project` com o campo `title` contendo 501 caracteres

Então: o sistema retorna HTTP 500 — Internal Server Error ou erro de validação

☐ (Aparecer onde o erro ocorreu)

Dado: que o usuário está autenticado

Quando: é enviada uma requisição POST para `/project` com o campo `title = "Projeto @#$% 2024!"`

Então: o sistema retorna HTTP 200 com `"Inserido com sucesso!"` (caracteres especiais devem ser aceitos em campo TEXT)

The screenshot shows a REST client interface with a PUT request to `https://sifsoft-funadm-server-production-a6f3.up.railway.app`. The request body is a JSON object with various fields including title, description, value, dates, instrument value, reserve fund value, coordinator name, coordinator email, bank account ID, category ID, status ID, project type work ID, author ID, and project number. The response is a 200 OK status with a response time of 323 ms and a body size of 516 B. The response body is a JSON object with a message, id, and a data object containing the same fields as the request body, with some values updated (e.g., id: 35, created_at: "2026-06-19T00:52:19.796Z").

```
PUT https://sifsoft-funadm-server-production-a6f3.up.railway.app Send 200 OK 323 ms 516 B
```

Params Body Auth Headers 5 Scripts Docs Preview Headers 11 Cookies Tests 0/0 → Mock

JSON Preview

```
1 {
2   "title": "##### TESTES API",
3   "description": "FOI INSERIDO VIA API - TESTES",
4   "value": 10,
5   "startDate": "2024-01-01T00:00:00Z",
6   "endDate": "2025-01-01T00:00:00Z",
7   "instrumentValue": 1,
8   "reservefundValue": 1,
9   "doaValue": 1,
10  "coordinatorName": "TEESTES API",
11  "coordinatorEmail": "TESTES.API@gmail.br",
12  "bankAccountId": 52,
13  "categoryId": 9,
14  "statusId": 3,
15  "projectTypeWorkId": 4,
16  "authorId": 2,
17  "projectNumber": 150000000
18 }
```

```
1 {
2   "message": "Projeto atualizado com sucesso!",
3   "id": 35,
4   "data": {
5     "id": 35,
6     "title": "##### TESTES API",
7     "description": "FOI INSERIDO VIA API - TESTES",
8     "value": "10",
9     "startDate": "2024-01-01T00:00:00.000Z",
10    "endDate": "2025-01-01T00:00:00.000Z",
11    "instrumentValue": "1",
12    "reservefundValue": "1",
13    "doaValue": "1",
14    "coordinatorName": "TEESTES API",
15    "coordinatorEmail": "TESTES.API@gmail.br",
16    "projectNumber": 150000000,
17    "bankAccountId": 52,
18    "authorId": 2,
19    "categoryId": 9,
20    "statusId": 3,
21    "projectTypeWorkId": 4,
22    "created_at": "2026-06-19T00:52:19.796Z"
23  }
24 }
```

#FalhouGrave #Falhou **CT_48: Cadastro com Data de Término Menor que Data de Início**

Dado: que o usuário está autenticado

Quando: é enviada uma requisição POST para `/project` com:

```
{
  "startDate": "2025-06-01T00:00:00Z",
  "endDate": "2024-01-01T00:00:00Z"
}
```

E: os demais campos estão válidos

Então: o sistema retorna erro de validação indicando que a data de término deve ser maior que a data de início

- ☐ (Não retornou erro e aceitou a requisição da 'Data de encerramento' ser antes da 'Data de Inicio')

PUT https://sifsoft-funadm-server-production-a6f3.up.railway.app/... Send 200 OK 469 ms 516 B Just Now

Params Body Auth Headers 5 Scripts Docs Preview Headers 11 Cookies Tests 0/0 → Mock Console

JSON

```
1 {
2   "title": "##### TESTES API",
3   "description": "FOI INSERIDO VIA API - TESTES",
4   "value": 10,
5   "startDate": "2025-01-01T00:00:00Z",
6   "endDate": "2024-01-01T00:00:00Z",
7   "instrumentValue": 1,
8   "reservefundValue": 1,
9   "doaValue": 1,
10  "coordinatorName": "TEESTES API",
11  "coordinatorEmail": "TESTES.API@gmail.br",
12  "bankAccountId": 52,
13  "categoryId": 9,
14  "statusId": 3,
15  "projectTypeWorkId": 4,
16  "authorId": 2,
17  "projectNumber": 150000000
18 }
```

Preview

```
1 {
2   "message": "Projeto atualizado com sucesso!",
3   "id": 35,
4   "data": {
5     "id": 35,
6     "title": "##### TESTES API",
7     "description": "FOI INSERIDO VIA API - TESTES",
8     "value": 10,
9     "startDate": "2025-01-01T00:00:00.000Z",
10    "endDate": "2024-01-01T00:00:00.000Z",
11    "instrumentValue": 1,
12    "reservefundValue": 1,
13    "doaValue": 1,
14    "coordinatorName": "TEESTES API",
15    "coordinatorEmail": "TESTES.API@gmail.br",
16    "projectNumber": 150000000,
17    "bankAccountId": 52,
18    "authorId": 2,
19    "categoryId": 9,
20    "statusId": 3,
21    "projectTypeWorkId": 4,
22    "created_at": "2026-06-19T00:52:19.796Z"
23  }
24 }
```

#Passou CT_49: Cadastro com Data de Início Igual à Data de Término

Dado: que o usuário está autenticado

Quando: é enviada uma requisição POST para /project com startDate igual a endDate

E: os demais campos estão válidos

Então: o sistema valida e aceita, conforme regra de negócio

PUT https://sifsoft-funadm-server-production-a6f3.up.railway.app 200 OK 348 ms 507 B Just Now

Params Body Auth Headers 5 Scripts Docs Preview Headers 11 Cookies Tests 0/0 → Mock Console

JSON Preview

```
1 {
2   "title": "TESTES API",
3   "description": "FOI INSERIDO VIA API - TESTES",
4   "value": 10,
5   "startDate": "2024-01-01T00:00:00Z",
6   "endDate": "2024-01-01T00:00:00Z",
7   "instrumentValue": 1,
8   "reservefundValue": 1,
9   "doaValue": 1,
10  "coordinatorName": "TEESTES API",
11  "coordinatorEmail": "TESTES.API@gmail.br",
12  "bankAccountId": 52,
13  "categoryId": 9,
14  "statusId": 3,
15  "projectTypeWorkId": 4,
16  "authorId": 2,
17  "projectNumber": 15000000
18 }
```

```
1 {
2   "message": "Projeto atualizado com sucesso!",
3   "id": 35,
4   "data": {
5     "id": 35,
6     "title": "TESTES API",
7     "description": "FOI INSERIDO VIA API - TESTES",
8     "value": "10",
9     "startDate": "2024-01-01T00:00:00.000Z",
10    "endDate": "2024-01-01T00:00:00.000Z",
11    "instrumentValue": "1",
12    "reservefundValue": "1",
13    "doaValue": "1",
14    "coordinatorName": "TEESTES API",
15    "coordinatorEmail": "TESTES.API@gmail.br",
16    "projectNumber": 15000000,
17    "bankAccountId": 52,
18    "authorId": 2,
19    "categoryId": 9,
20    "statusId": 3,
21    "projectTypeWorkId": 4,
22    "created_at": "2026-06-19T00:52:19.796Z"
23  }
```

#Passou CT_50: Cadastro com Valor do Instrumento Grande (NUMERIC 16,2)

Dado: que o usuário está autenticado

Quando: é enviada uma requisição POST para /project com instrumentValue = "99999999999999.99" (14 dígitos inteiros, 2 decimais)

Então: o sistema retorna HTTP 200 com "Inserido com sucesso!"

PUT https://sifsoft-funadm-server-production-a6f3.up.railway.app 200 OK 327 ms 523 B Just Now

Params Body Auth Headers 5 Scripts Docs Preview Headers 11 Cookies Tests 0/0 → Mock Console

JSON Preview

```
1 {
2   "title": "TESTES API",
3   "description": "FOI INSERIDO VIA API - TESTES",
4   "value": 10,
5   "startDate": "2024-01-01T00:00:00Z",
6   "endDate": "2024-01-01T00:00:00Z",
7   "instrumentValue": 99999999999999.99,
8   "reservefundValue": 1,
9   "doaValue": 1,
10  "coordinatorName": "TEESTES API",
11  "coordinatorEmail": "TESTES.API@gmail.br",
12  "bankAccountId": 52,
13  "categoryId": 9,
14  "statusId": 3,
15  "projectTypeWorkId": 4,
16  "authorId": 2,
17  "projectNumber": 15000000
18 }
```

```
1 {
2   "message": "Projeto atualizado com sucesso!",
3   "id": 35,
4   "data": {
5     "id": 35,
6     "title": "TESTES API",
7     "description": "FOI INSERIDO VIA API - TESTES",
8     "value": "10",
9     "startDate": "2024-01-01T00:00:00.000Z",
10    "endDate": "2024-01-01T00:00:00.000Z",
11    "instrumentValue": "99999999999999.98",
12    "reservefundValue": "1",
13    "doaValue": "1",
14    "coordinatorName": "TEESTES API",
15    "coordinatorEmail": "TESTES.API@gmail.br",
16    "projectNumber": 15000000,
17    "bankAccountId": 52,
18    "authorId": 2,
19    "categoryId": 9,
20    "statusId": 3,
21    "projectTypeWorkId": 4,
22    "created_at": "2026-06-19T00:52:19.796Z"
23  }
```

#Sugestão

#Passou

CT_51: Cadastro com Valor do Instrumento Nulo

Dado: que o usuário está autenticado

Quando: é enviada uma requisição POST para `/project` com `instrumentValue = null`

Então: o sistema retorna HTTP 500 ou erro de validação de campo obrigatório

☐ (Mostrar qual erro ocorreu)

PUT `https://sifsoft-funadm-server-production-a6f3.up.railway.app` Send

500 Internal Server Error 348 ms 41 B Just Now

Params Body Auth Headers 5 Scripts Docs Preview Headers 10 Cookies Tests 0/0 → Mock Console

JSON

```
1 {
2   "title": "TESTES API",
3   "description": "FOI INSERIDO VIA API - TESTES",
4   "value": 10,
5   "startDate": "2024-01-01T00:00:00Z",
6   "endDate": "2024-01-01T00:00:00Z",
7   "instrumentValue": null,
8   "reservefundValue": 1,
9   "doaValue": 1,
10  "coordinatorName": "TESTES API",
11  "coordinatorEmail": "TESTES.API@gmail.br",
12  "bankAccountId": 52,
13  "categoryId": 9,
14  "statusId": 3,
15  "projectTypeWorkId": 4,
16  "authorId": 2,
17  "projectNumber": 150000000
18 }
```

Preview

```
1 {
2   "message": "Error ao atualizar Projeto!"
3 }
```

#Falhou

CT_52: Cadastro com E-mail do Coordenador Sem Formato Válido

Dado: que o usuário está autenticado

Quando: é enviada uma requisição POST para `/project` com `coordinatorEmail = "emailinvalido"`

E: os demais campos estão válidos

Então: o sistema retorna erro de validação de formato de e-mail

☐ (Pela API foi aceito o campo coordinatorEmail sem a formatação ideal)

PUT https://sifsoft-funadm-server-production-a6f3.up.railway.app/ Send 200 OK 315 ms 499 B Just Now

Params Body Auth Headers 5 Scripts Docs Preview Headers 11 Cookies Tests 0/0 → Mock Console

JSON Preview

```
1 {
2   "title": "TESTES API",
3   "description": "FOI INSERIDO VIA API - TESTES",
4   "value": 10,
5   "startDate": "2024-01-01T00:00:00Z",
6   "endDate": "2024-01-01T00:00:00Z",
7   "instrumentValue": 10,
8   "reservefundValue": 1,
9   "doaValue": 1,
10  "coordinatorName": "TEESTES API",
11  "coordinatorEmail": "TESTES.API",
12  "bankAccountId": 52,
13  "categoryId": 9,
14  "statusId": 3,
15  "projectTypeWorkId": 4,
16  "authorId": 2,
17  "projectNumber": 150000000
18 }
```

```
1 {
2   "message": "Projeto atualizado com sucesso!",
3   "id": 35,
4   "data": {
5     "id": 35,
6     "title": "TESTES API",
7     "description": "FOI INSERIDO VIA API - TESTES",
8     "value": "10",
9     "startDate": "2024-01-01T00:00:00.000Z",
10    "endDate": "2024-01-01T00:00:00.000Z",
11    "instrumentValue": "10",
12    "reservefundValue": "1",
13    "doaValue": "1",
14    "coordinatorName": "TEESTES API",
15    "coordinatorEmail": "TESTES.API",
16    "projectNumber": 150000000,
17    "bankAccountId": 52,
18    "authorId": 2,
19    "categoryId": 9,
20    "statusId": 3,
21    "projectTypeWorkId": 4,
22    "created_at": "2026-06-19T00:52:19.796Z"
23   }
24 }
```

#FalhouGrave #Falhou CT_53: Cadastro com Conta Bancária Inexistente (bankAccountId inválido)

Dado: que o usuário está autenticado

Quando: é enviada uma requisição POST para /project com bankAccountId = 9999 (não existente)

E: os demais campos estão válidos

Então: o sistema retorna HTTP 500 — Internal Server Error por violação de chave estrangeira

☐ (Não existe esse bankAccountID e a API aceitou essa notificação)

POST -production-a6f3.up.railway.app/bankAccount/99999 Send 200 OK 302 ms 26 B Just Now

Params Body Auth Headers 5 Scripts Docs Preview Headers 10 Cookies Tests 0/0 → Mock Console

JSON Preview

```
1 {
2   "title": "TESTES API",
3   "description": "FOI INSERIDO VIA API - TESTES",
4   "value": 10,
5   "startDate": "2024-01-01T00:00:00Z",
6   "endDate": "2024-01-01T00:00:00Z",
7   "instrumentValue": 10,
8   "reservefundValue": 1,
9   "doaValue": 1,
10  "coordinatorName": "TEESTES API",
11  "coordinatorEmail": "TESTES.API",
12  "bankAccountId": 99999,
13  "categoryId": 9,
14  "statusId": 3,
15  "projectTypeWorkId": 4,
16  "authorId": 2,
17  "projectNumber": 150000000
18 }
```

```
1 {
2   "message": "Hello World!"
3 }
```

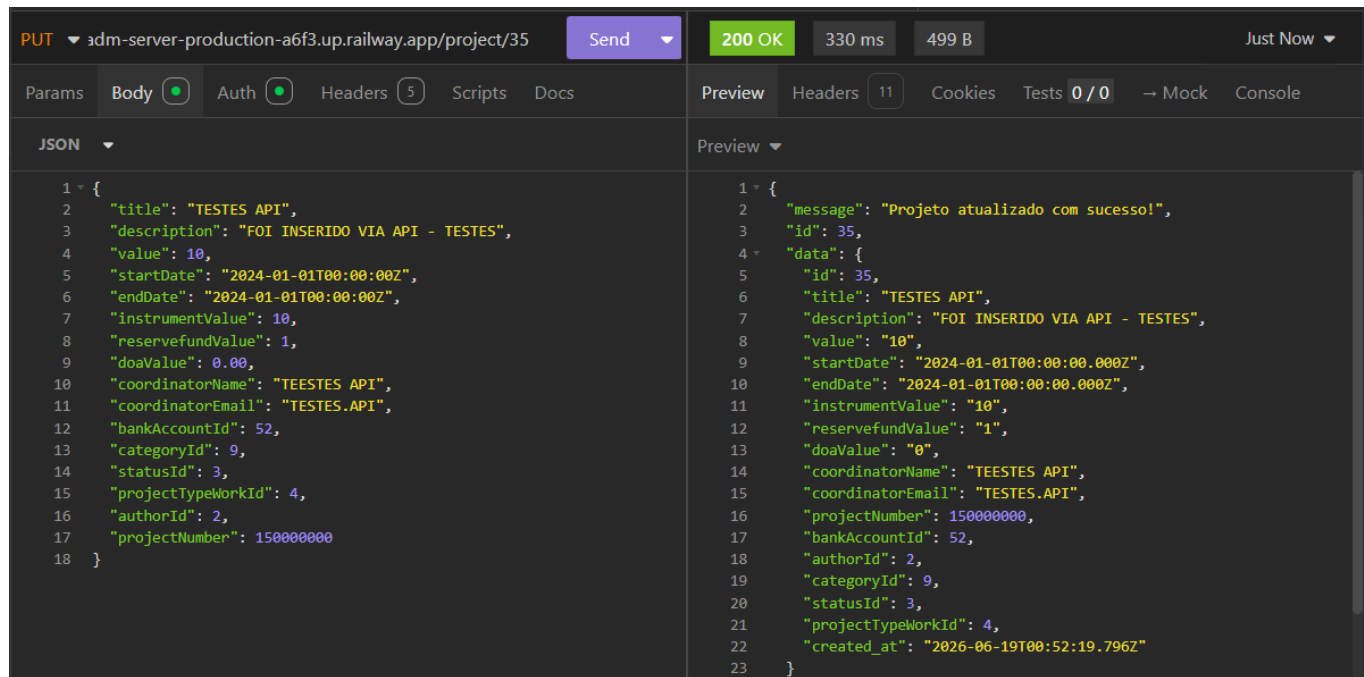
#Passou CT_54: Cadastro com Porcentagem DOA Zerada (0,00)

Dado: que o usuário está autenticado

Quando: é enviada uma requisição POST para `/project` com `doaValue = "0.00"`

E: os demais campos estão válidos

Então: o sistema retorna HTTP 200 com `"Inserido com sucesso!"`



#Sugestão #Passou CT_55: Cadastro com Porcentagem DOA Nula

Dado: que o usuário está autenticado

Quando: é enviada uma requisição POST para `/project` com `doaValue = null`

Então: o sistema retorna HTTP 500 ou erro de validação

☐ (Mostrar onde ocorreu o erro)

PUT https://sifsoft-funadm-server-production-a6f3.up.railway.app 500 Internal Server Error 323 ms 41 B Just Now

Params Body Auth Headers 5 Scripts Docs Preview Headers 10 Cookies Tests 0/0 → Mock Console

JSON

```
1 {
2   "title": "TESTES API",
3   "description": "FOI INSERIDO VIA API - TESTES",
4   "value": 10,
5   "startDate": "2024-01-01T00:00:00Z",
6   "endDate": "2024-01-01T00:00:00Z",
7   "instrumentValue": 10,
8   "reservefundValue": 1,
9   "doaValue": null,
10  "coordinatorName": "TEESTES API",
11  "coordinatorEmail": "TESTES.API",
12  "bankAccountId": 52,
13  "categoryId": 9,
14  "statusId": 3,
15  "projectTypeWorkId": 4,
16  "authorId": 2,
17  "projectNumber": 150000000
18 }
```

Preview

```
1 {
2   "message": "Error ao atualizar Projeto!"
3 }
```

Projetos — Listagem e Busca (API — GET /project e GET /project/{id})

#Passou CT_56: Listar Todos os Projetos com Token Válido

Dado: que o usuário está autenticado

Quando: é enviada uma requisição GET para /project

Então: o sistema retorna HTTP 200 com a lista de projetos cadastrados

GET funadm-server-production-a6f3.up.railway.app/project 200 OK 470 ms 31.8 KB Just Now

Params Body Auth Headers 5 Scripts Docs Preview Headers 11 Cookies Tests 0/0 → Mock Console

JSON

```
1 ...
```

Preview

```
281 },
282 {
283   "id": 18,
284   "title": "Teste",
285   "description": "teste",
286   "value": "3.82",
287   "startDate": "2024-08-05T03:00:00.000Z",
288   "endDate": "2026-07-30T03:00:00.000Z",
289   "instrumentValue": "262000.31",
290   "reservefundValue": "1000",
291   "doaValue": "10000",
292   "coordinatorName": "Bruna Castro Porto",
293   "coordinatorEmail": "teste@mail.com",
294   "projectNumber": 21515161,
295   "bankAccountId": 32,
296   "authorId": 2,
297   "categoryId": 9,
298   "statusId": 3,
299   "projectTypeWorkId": 4,
300   "created_at": "2025-08-24T15:36:26.575Z"
301 }
```


#Passou CT_57: Buscar Projeto por ID Existente

Dado: que existe um projeto com `id = 35` e o usuário está autenticado

Quando: é enviada uma requisição GET para `/project/35`

Então: o sistema retorna HTTP 200 com os dados completos do projeto

The screenshot shows a REST client interface with the following details:

- Method: GET
- URL: `adm-server-production-a6f3.up.railway.app/project/35`
- Status: 200 OK
- Time: 312 ms
- Size: 438 B
- Body: JSON
- Preview:

```
1 {
2   "id": 35,
3   "title": "TESTES API",
4   "description": "FOI INSERIDO VIA API - TESTES",
5   "value": "10",
6   "startDate": "2024-01-01T00:00:00.000Z",
7   "endDate": "2024-01-01T00:00:00.000Z",
8   "instrumentValue": "10",
9   "reservefundValue": "1",
10  "doaValue": "0",
11  "coordinatorName": "TEESTES API",
12  "coordinatorEmail": "TESTES.API",
13  "projectNumber": 150000000,
14  "bankAccountId": 52,
15  "authorId": 2,
16  "categoryId": 9,
17  "statusId": 3,
18  "projectTypeWorkId": 4,
19  "created_at": "2026-06-19T00:52:19.796Z"
20 }
```

#Passou CT_58: Buscar Projeto por ID Inexistente

Dado: que o usuário está autenticado

Quando: é enviada uma requisição GET para `/project/9999`

Então: o sistema retorna HTTP 404 — Not Found

The screenshot shows a REST client interface with the following details:

- Method: GET
- URL: `i-server-production-a6f3.up.railway.app/project/9999`
- Status: 404 Not Found
- Time: 293 ms
- Size: 38 B
- Body: JSON
- Preview:

```
1 {
2   "message": "Projeto não encontrado!"
3 }
```

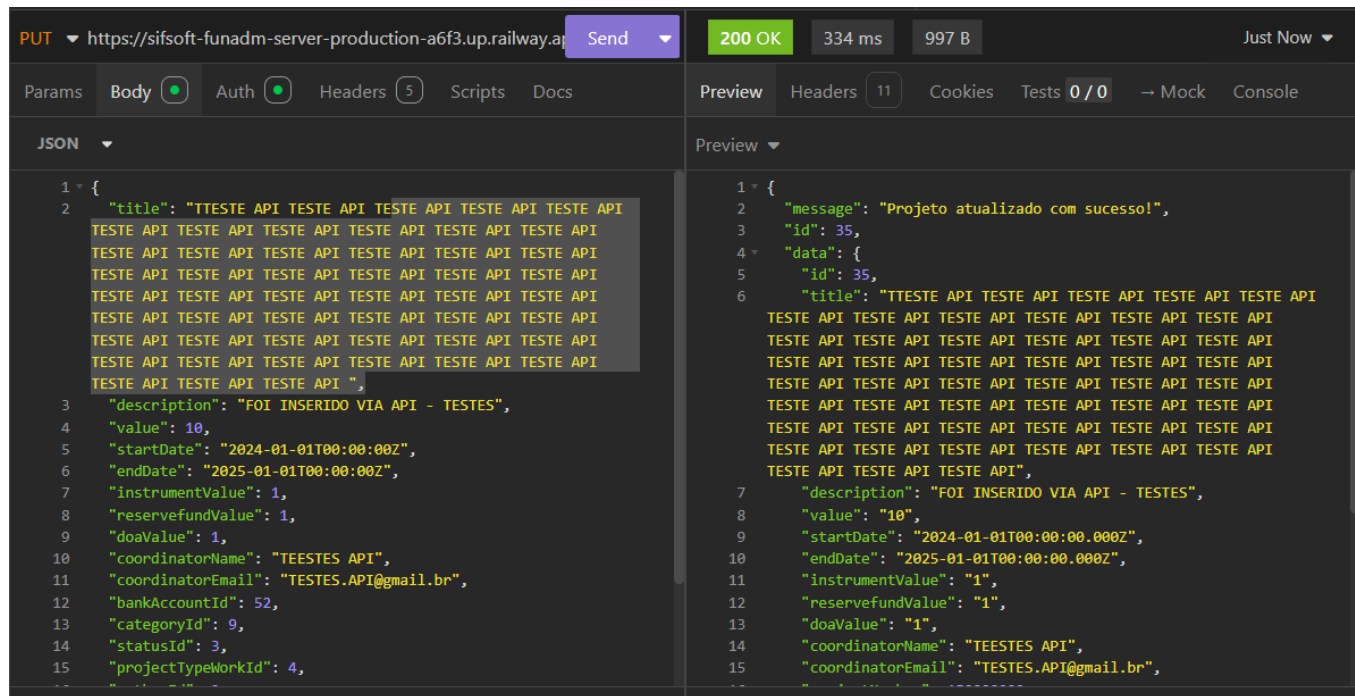
Projetos — Edição (API — PUT `/project/{id}`)

#Passou CT_59: Editar Campos Editáveis do Projeto com Dados Válidos

Dado: que existe um projeto com `id = 35` e o usuário está autenticado

Quando: é enviada uma requisição PUT para `/project/35` com o body:

Então: o sistema retorna HTTP 200 e os dados são atualizados



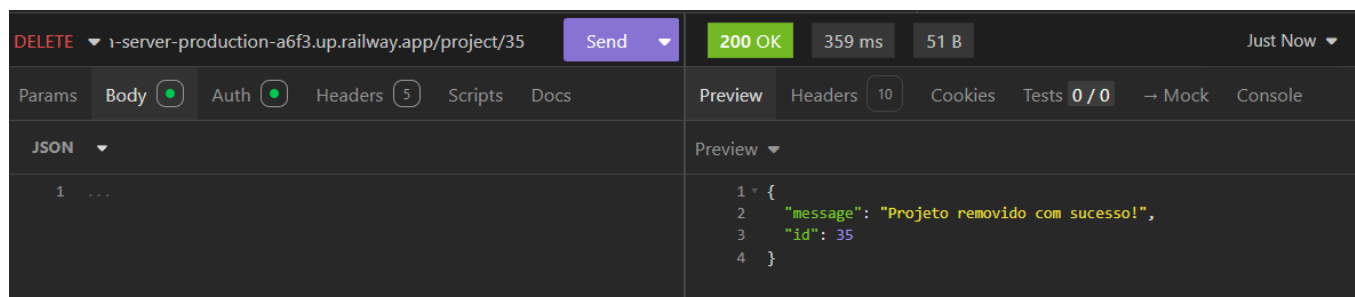
Projetos — Exclusão (API — DELETE `/project/{id}`)

#Passou CT_60: Excluir Projeto Existente sem Dependências

Dado: que existe um projeto com `id = 35` sem despesas vinculadas e o usuário está autenticado

Quando: é enviada uma requisição DELETE para `/project/35`

Então: o sistema retorna HTTP 200 com mensagem de sucesso

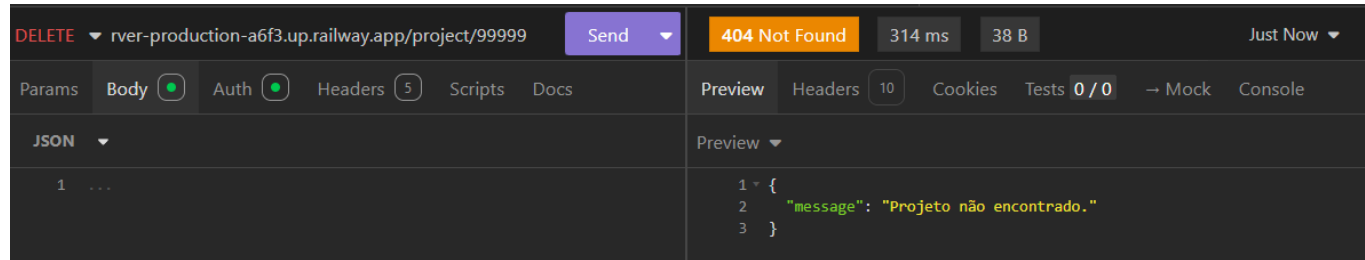


#Passou CT_61: Excluir Projeto com ID Inexistente

Dado: que o usuário está autenticado

Quando: é enviada uma requisição DELETE para `/project/9999`

Então: o sistema retorna HTTP 404 — Not Found ou HTTP 500



Projetos — Tela (Frontend)

#Passou CT_62: Verificar Conta Bancária Cadastrada Aparece no Select do Formulário

Dado: que existe uma conta bancária cadastrada com tipo diferente de "Projeto"

E: o usuário acessa o formulário de cadastro de projeto

Quando: o usuário clica no campo "Conta Bancária"

Então: a conta bancária cadastrada não aparece como opção disponível no select

(Aparece apenas a conta bancária do tipo 'Projeto')



#Passou CT_63: Verificar Máscara do Campo "Valor do Instrumento" (1.000.000,00)

Dado: que o usuário está no formulário de cadastro de projeto

Quando: o usuário digita 1000000 no campo "Valor do Instrumento"

Então: o sistema exibe o valor formatado como 1.000.000,00

Valor do instrumento: *

100.000,00

#Passou CT_64: Verificar Máscara do Campo "Porcentagem DOA" (10,00)

Dado: que o usuário está no formulário de cadastro de projeto

Quando: o usuário digita 10 no campo "Porcentagem da DOA"

Então: o sistema exibe o valor formatado como 10,00

Valor da DOA: *

10,00

#Passou CT_65: Verificar Máscara do Campo "Data de Início" (dd/mm/aaaa)

Dado: que o usuário está no formulário de cadastro de projeto

Quando: o usuário digita uma data no campo "Data de Início"

Então: o sistema aplica a máscara dd/mm/aaaa automaticamente

(A máscara só funciona quando clica na data no calendário, caso digite direto como '17062026' a máscara não é aplicada)

Data de início: *

17/06/2026

#Passou CT_66: Verificar Máscara do Campo "Data de término" (dd/mm/aaaa)

Dado: que o usuário está no formulário de cadastro de projeto

Quando: o usuário digita uma data no campo "Data de término"

Então: o sistema aplica a máscara dd/mm/aaaa automaticamente

(A máscara só funciona quando clica na data no calendário, caso digite direto como '17062026' a máscara não é aplicada)

Data de término: *

29/06/2026

#Passou CT_67: Status do Projeto Seleccionável no Cadastro

Dado: que o usuário está no formulário de cadastro de projeto

Quando: o usuário acessa o campo "Status do Projeto"

Então: as opções de status disponíveis são exibidas para seleção

Nome AZ
Finalizado
Em fase de prestação de contas
Encerrado
Ainda não iniciou
Em andamento

Status do projeto: *

Pesquisar conta bancária...

Ainda não iniciou

Em andamento

Em fase de prestação de contas

Encerrado

Finalizado

#Falhou CT_68: Botão Limpar no Formulário de Cadastro de Projeto

Dado: que o usuário preencheu todos os campos do formulário de cadastro de projeto

Quando: o usuário clica no botão "Limpar"

Então: todos os campos são redefinidos ao estado inicial

☐ (O botão 'Limpar' não apaga as Datas)

Nº: *

Digite o numero...

Projeto: *

Digite o Título...

Data de início: *

04/06/2026

Data de término: *

→ 14/07/2026



Tipo no plano de trabalho: *

selecione o tipo...



Modalidade: *

Selecione o tipo de projeto...



Conta bancária: *

Pesquisar conta bancária...



Descrição: *

Digite a descrição...

Valor do instrumento: *

Digite o valor...

Valor da DOA: *

Digite o valor da doa...

% da DOA sobre o valor do instrumento:

Aguardando...

Valor do fundo de reserva:

Aguardando...



Status do projeto: *

Pesquisar conta bancária...



Nome do coordenador: *

Digite o nome...

E-mail do coordenador: *

Digite o e-mail...

Limpar

Cadastrar

#Passou CT_69: Nome do Coordenador com Acentos e Caracteres Especiais

Dado: que o usuário está no formulário de cadastro de projeto

Quando: o usuário preenche o campo "Nome do Coordenador" com "José Antônio Çoão"

Então: o sistema aceita o valor e exibe corretamente em tela

Nome do coordenador: *

José Antônio Çoão

#Passou CT_70: E-mail do Coordenador com Formato Inválido na Tela

Dado: que o usuário está no formulário de cadastro de projeto

Quando: o usuário preenche o campo "E-mail do Coordenador" com "email_invalido"

E: clica em "Cadastrar"

Então: o sistema exibe mensagem de validação indicando formato de e-mail inválido e não prossegue com o cadastro

E-mail do coordenador: *

JOSE.COM|



Inclua um "@" no endereço de e-mail. "JOSE.COM" está com um "@" faltando.

PARTE 2

Movimentações — Saídas — Cadastro (API — POST /expense)

#Passou CT_71: Cadastro de Movimentação com Todos os Campos Obrigatórios Válidos

Dado: que o usuário está autenticado e existem projeto e conta bancária cadastrados

Quando: é enviada uma requisição POST para `/expense` com o body certo

Então: o sistema retorna HTTP 200 com "Inserido com sucesso!" e o id da movimentação criada

#Sugestão #Passou **CT_72: Cadastro com Título Nulo (Campo Obrigatório)**

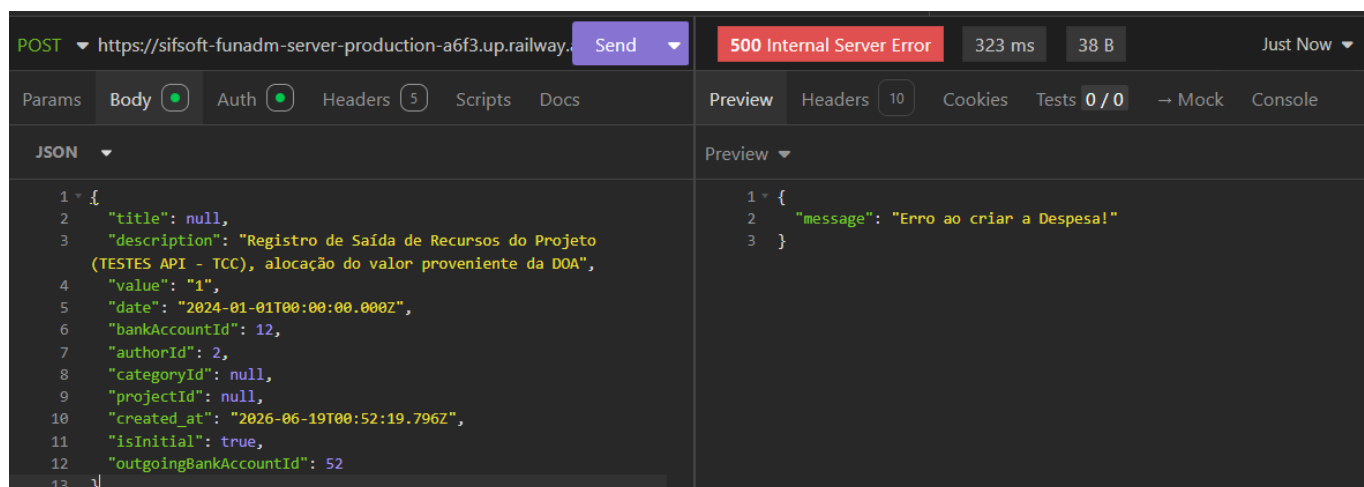
Dado: que o usuário está autenticado

Quando: é enviada uma requisição POST para `/expense` com o campo `title` como `null`

E: os demais campos obrigatórios estão preenchidos com valores válidos

Então: o sistema retorna HTTP 500 — Internal Server Error ou erro de campo obrigatório

☐ (Mostrar onde está o erro)



#Sugestão #Passou **CT_73: Cadastro com Valor Nulo (Campo Obrigatório)**

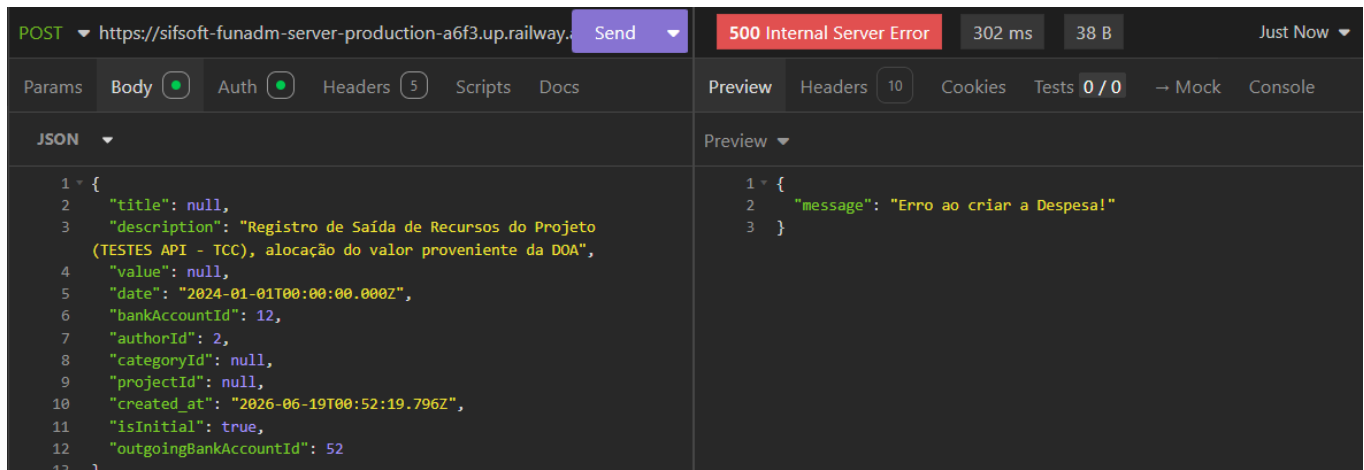
Dado: que o usuário está autenticado

Quando: é enviada uma requisição POST para `/expense` com o campo `value` como `null`

E: os demais campos obrigatórios estão preenchidos com valores válidos

Então: o sistema retorna HTTP 500 — Internal Server Error

☐ (Não é mostrado onde está o erro)



#Sugestão #Passou CT_74: Cadastro com Data Nula (Campo Obrigatório)

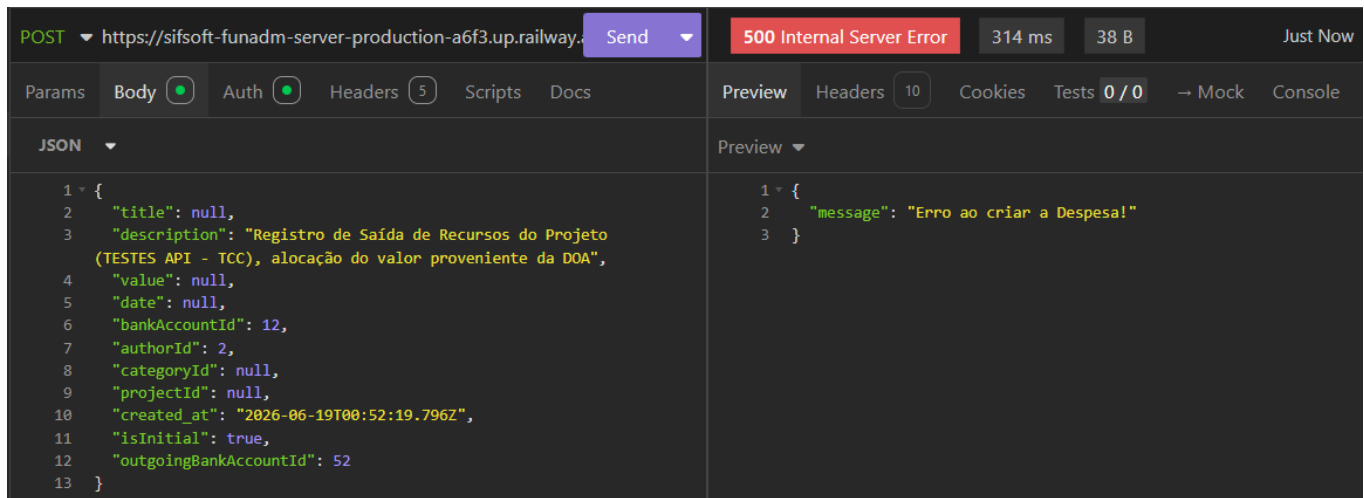
Dado: que o usuário está autenticado

Quando: é enviada uma requisição POST para `/expense` com o campo `date` como `null`

E: os demais campos obrigatórios estão preenchidos com valores válidos

Então: o sistema retorna HTTP 500 — Internal Server Error

☐ (Não é mostrado onde está o erro)



#Falhou CT_75: Cadastro com Valor Negativo

Dado: que o usuário está autenticado

Quando: é enviada uma requisição POST para `/expense` com `value` = `"-100.00"`

E: os demais campos obrigatórios estão preenchidos com valores válidos

Então: o sistema retorna erro de validação ou HTTP 500 (valor de pagamento não deve ser negativo)

☐ (Aceitou o valor negativo)

#Sugestão #Passou **CT_76: Cadastro com Valor em Formato Inválido (Letras)**

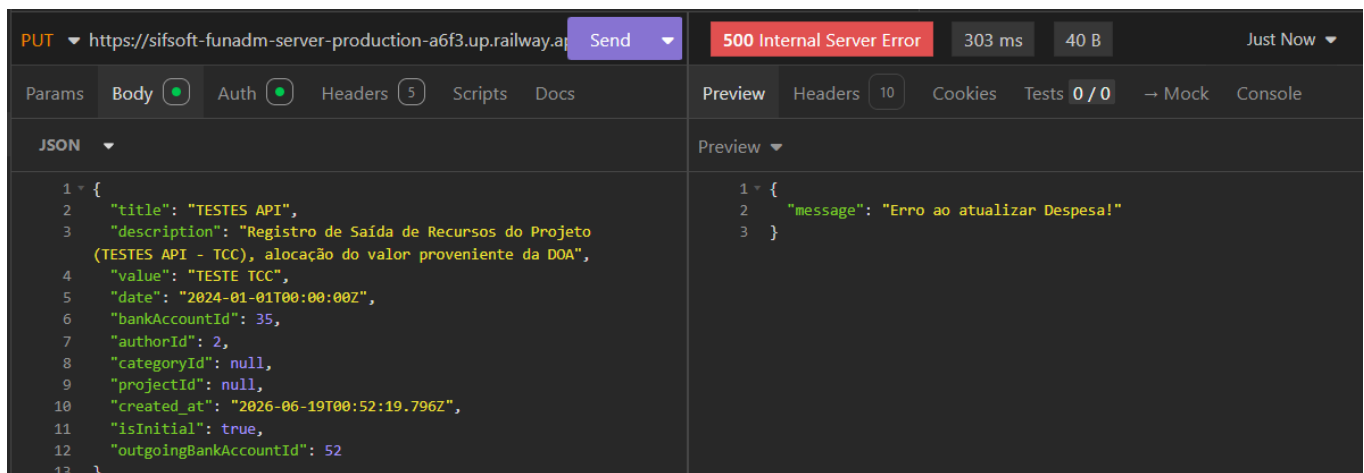
Dado: que o usuário está autenticado

Quando: é enviada uma requisição POST para `/expense` com `value = "abc"`

E: os demais campos obrigatórios estão preenchidos com valores válidos

Então: o sistema retorna HTTP 500 — Internal Server Error por tipo de dado inválido

☐ (Mostrar onde está o erro)



#Passou **CT_77: Cadastro com Data em Formato Inválido**

Dado: que o usuário está autenticado

Quando: é enviada uma requisição POST para `/expense` com `date = "32/13/2024"`

E: os demais campos obrigatórios estão preenchidos com valores válidos

Então: o sistema retorna HTTP 500 — Internal Server Error ou erro de formato de data

#Passou **CT_78: Cadastro com Data Contendo Caracteres Especiais**

Dado: que o usuário está autenticado

Quando: é enviada uma requisição POST para `/expense` com `date = "@@/##/$$$$"`

E: os demais campos obrigatórios estão preenchidos com valores válidos

Então: o sistema retorna HTTP 500 — Internal Server Error

#Sugestão #Passou **CT_79: Cadastro com bankAccountId Inexistente**

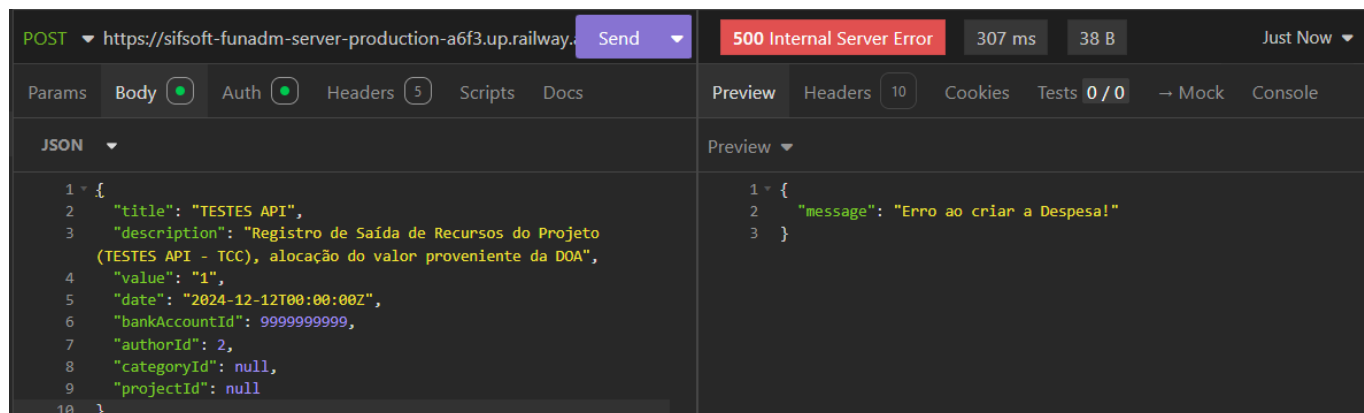
Dado: que o usuário está autenticado

Quando: é enviada uma requisição POST para `/expense` com `bankAccountId = 9999`

E: os demais campos obrigatórios estão preenchidos com valores válidos

Então: o sistema retorna HTTP 500 — Internal Server Error por violação de chave estrangeira

☐ (Mostrar onde foi o erro)

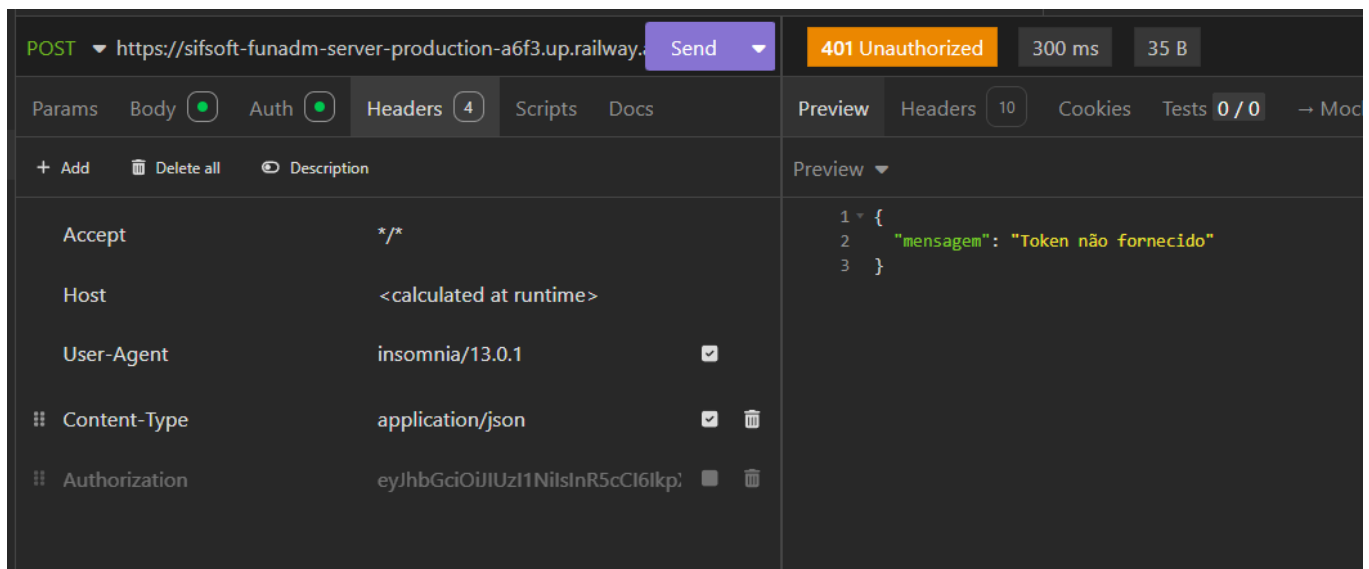


#Passou **CT_80: Cadastro sem Token de Autenticação**

Dado: que o usuário não está autenticado

Quando: é enviada uma requisição POST para `/expense` com body válido e sem o header `Authorization`

Então: o sistema retorna HTTP 401 — Unauthorized



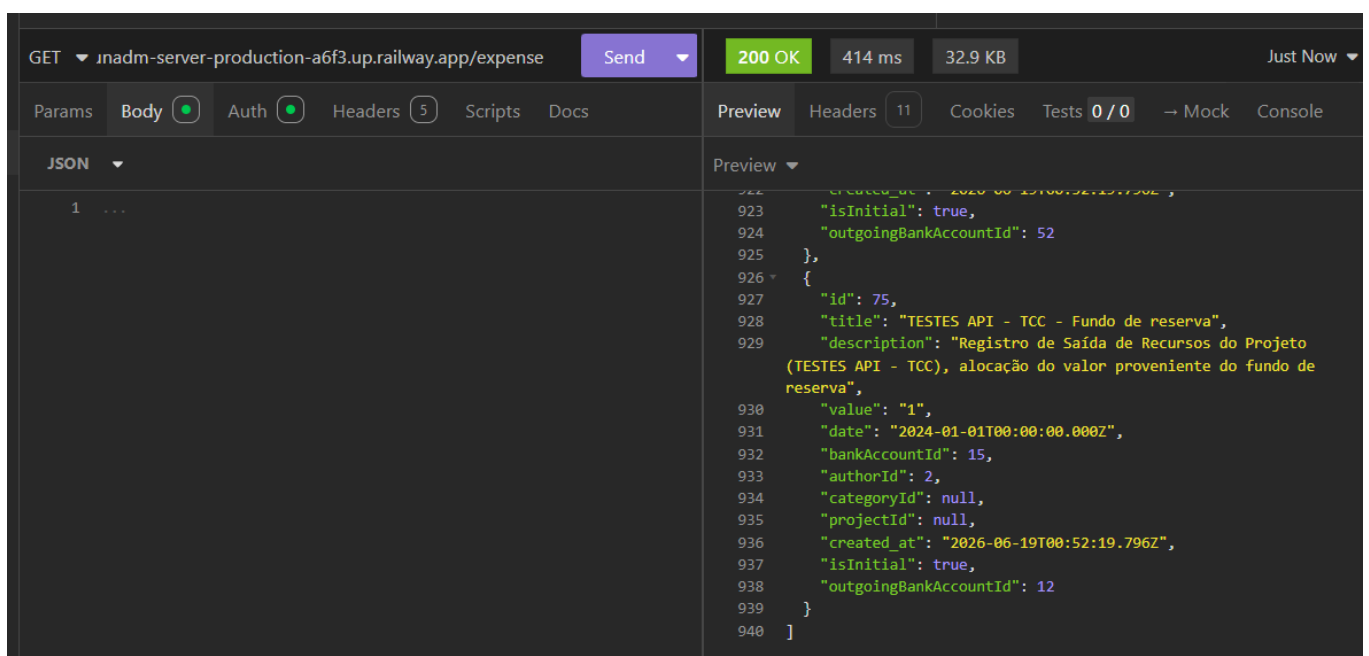
Movimentações — Saídas — Listagem e Busca (GET /expense e GET /expense/{id})

#Passou CT_81: Listar Todas as Movimentações com Token Válido

Dado: que o usuário está autenticado

Quando: é enviada uma requisição GET para /expense

Então: o sistema retorna HTTP 200 com a lista de despesas cadastradas

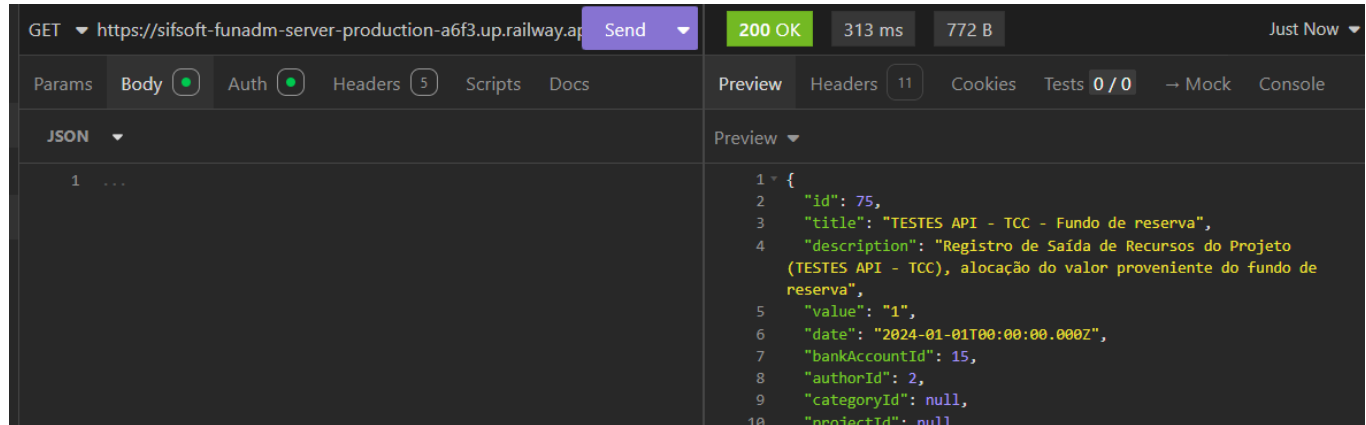


#Passou CT_82: Buscar Movimentação por ID Existente

Dado: que existe uma despesa com `id = 1` e o usuário está autenticado

Quando: é enviada uma requisição GET para `/expense/1`

Então: o sistema retorna HTTP 200 com os dados completos da despesa

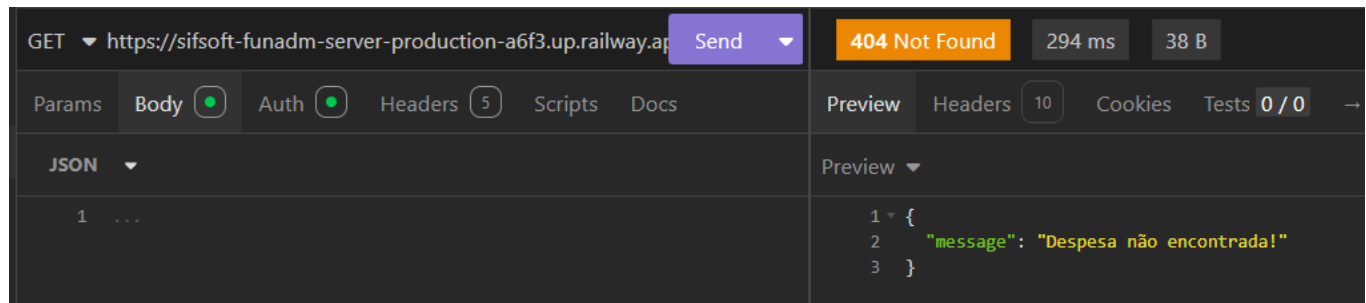


#Passou CT_83: Buscar Movimentação por ID Inexistente

Dado: que o usuário está autenticado

Quando: é enviada uma requisição GET para `/expense/9999`

Então: o sistema retorna HTTP 404 — Not Found



Movimentações — Saídas — Edição (PUT `/expense/{id}`)

#Passou CT_84: Editar Movimentação com Dados Válidos

Dado: que existe uma despesa com `id = 589` e o usuário está autenticado

Quando: é enviada uma requisição PUT para `/expense/589` com o body:

```
{
  "title": "Pagamento Atualizado - Teste TCC",
  "value": "3000.00",
  "date": "2024-07-01T00:00:00Z",
  "bankAccountId": 31,
  "categoryId": null,
  "authorId": 1
}
```

Então: o sistema retorna HTTP 200 e os dados da despesa são atualizados

The screenshot shows a REST client interface with a PUT request to `https://cash-management-teste.up.railway.app/expense/589`. The status is **200 OK** with a response time of 720 ms and 402 B of data. The request body is a JSON object. The response preview shows a JSON object with the updated expense details, including an ID of 589, a description, and a creation timestamp.

```
PUT https://cash-management-teste.up.railway.app/expense/589 Send 200 OK 720 ms 402 B Just Now
```

Params Body Auth Headers 4 Scripts Docs Preview Headers 10 Cookies Tests 0/0 → Mock Console

JSON

```
1 {
2   "title": "Pagamento Atualizado - Teste TCC",
3   "value": 3000.00,
4   "date": "2024-07-01T00:00:00Z",
5   "bankAccountId": 31,
6   "categoryId": null,
7   "authorId": 1
8 }
```

Preview

```
1 {
2   "id": 589,
3   "title": "Pagamento Atualizado - Teste TCC",
4   "description": "Registro de Saída de Recursos do Projeto
5   Projeto Alvorada: Inclusão social e produtiva de pessoas egressas
6   do S, incluindo a alocação do valor proveniente do Fundo de
7   Reserva.",
8   "value": 3000,
9   "date": "2024-07-01T00:00:00.000Z",
10  "bankAccountId": 31,
11  "authorId": 1,
12  "categoryId": null,
13  "projectId": 307,
14  "created_at": "2024-07-17T17:23:27.785Z"
15 }
```

#Sugestão #Passou CT_85: Editar Movimentação com ID Inexistente

Dado: que o usuário está autenticado

Quando: é enviada uma requisição PUT para `/expense/9999` com body válido

Então: o sistema retorna HTTP 500 — Internal Server Error ou 404

☐ (Mostrar qual o erro)

The screenshot shows a REST client interface with a PUT request to `https://sifsoft-funadm-server-production-a6f3.up.railway.app/expense/9999`. The status is **500 Internal Server Error** with a response time of 1.38 s and 40 B of data. The request headers are visible. The response preview shows a JSON object with an error message.

```
PUT https://sifsoft-funadm-server-production-a6f3.up.railway.app/expense/9999 Send 500 Internal Server Error 1.38 s 40 B Just Now
```

Params Body Auth Headers 5 Scripts Docs Preview Headers 10 Cookies Tests 0/0 → Mock Console

+ Add Delete all Description

Accept */*

Host <calculated at runtime>

User-Agent insomnia/13.0.1

Preview

```
1 {
2   "message": "Erro ao atualizar Despesa!"
3 }
```

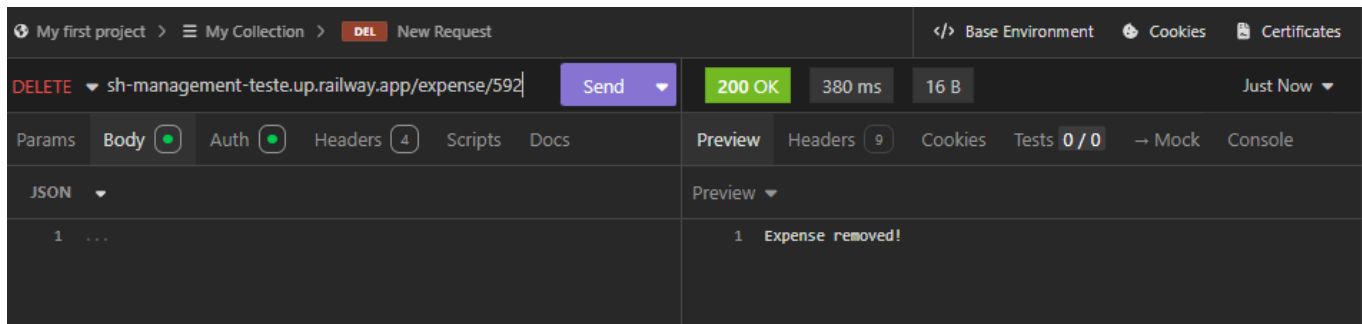
Movimentações — Saídas — Exclusão (DELETE /expense/{id})

#Passou CT_86: Excluir Movimentação Existente

Dado: que existe uma despesa com `id = 592` e o usuário está autenticado

Quando: é enviada uma requisição DELETE para `/expense/592`

Então: o sistema retorna HTTP 200 com mensagem de sucesso



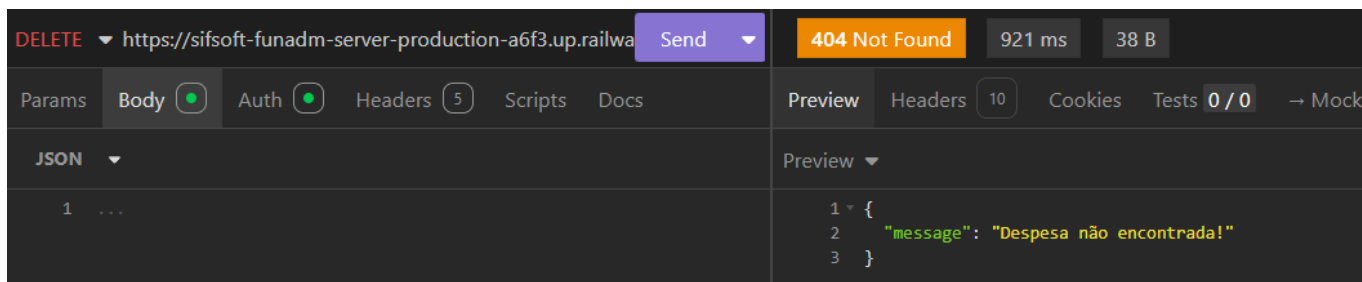
#Sugestão #Passou CT_87: Excluir Movimentação com ID Inexistente

Dado: que o usuário está autenticado

Quando: é enviada uma requisição DELETE para `/expense/9999`

Então: o sistema retorna HTTP 500 — Internal Server Error ou 404

☐ (Mostrar o erro)



Movimentações — Saídas — Tela (Frontend)

#Passou CT_88: Verificar Máscara do Campo "Data" no Formulário (dd/mm/aaaa)

Dado: que o usuário está no formulário de cadastro de movimentação de saída

Quando: o usuário digita uma data no campo "Data"

Então: o sistema aplica a máscara dd/mm/aaaa automaticamente

Data: *

10/01/3012

#Passou CT_89: Selecionar Projeto Atualiza Automaticamente o Campo "Conta Debitada"

Dado: que o usuário está no formulário de cadastro de movimentação de saída

E: existe um projeto com conta bancária vinculada

Quando: o usuário seleciona um projeto no campo "Projeto"

Então: o campo "Conta Debitada" é atualizado automaticamente para refletir a conta vinculada ao projeto escolhido

Título: *

pgnt 2

Tipo de pagamento: *

Pagamento de salário

+

Projeto: *

5 - Capacitação em Metodologias Ativas para os Professores da Educação Básica

Conta Debitada: *

Nome: Banco do Brasil - Agência: 0976-8 - N°: 37911-5

Saldo da conta bancária:

Valor: R\$ 2.893,10

Descrição: *

jghngh

Valor do pagamento: *

50,00

Data: *

12/11/2025

Voltar

Salvar

Título: *		Tipo de pagamento: *	
pgnt 2		Pagamento de salário	
Projeto: *		Descrição: *	
Teste		jghngh	
Conta Debitada: *			
Nome: TESTE NÚMERO 02 - Agência: 0912 - N°: 1231665			
Saldo da conta bancária:			
Valor:		R\$ 152.000,31	
Valor do pagamento: *		Data: *	
50,00		12/11/2025	
		Voltar	
		Salvar	

#Passou CT_90: Botão 'Limpar' Esvazia Todos os Campos do Formulário de Movimentação

Dado: que o usuário preencheu todos os campos do formulário de movimentação de saída

Quando: o usuário clica no botão "Limpar"

Então: todos os campos do formulário são redefinidos ao estado inicial

Título: *		Tipo de pagamento: *	
TESTE		Pagamento de salário	
Projeto:		Descrição: *	
Teste		TESTE	
Conta Debitada: *			
Nome: TESTE NÚMERO 02 - Agência: 0912 - N°: 1231665			
Saldo da conta bancária:			
Valor:		R\$ 152.000,31	
Valor do pagamento: *		Data: *	
10,00		31/12/2020	
		Limpar	
		Cadastrar	

Título: *
Digite o Título...

Tipo de pagamento: *
Selecione o tipo de projeto... ▾

Projeto:
Pesquisar projeto... ▾

Conta Debitada: *
Selecione conta bancária... ▾

Saldo da conta bancária:
Valor: R\$ 0,00

Descrição: *
Digite a descrição...

Valor do pagamento: *
Digite o valor...

Data: *
dd/mm/aaaa

Limpar

Cadastrar

#Passou CT_91: Botão 'Cadastrar' Registra Movimentação com Campos Obrigatórios Válidos

Dado: que o usuário preencheu corretamente os campos obrigatórios (Título, Projeto, Valor e Data)

Quando: o usuário clica no botão "Cadastrar"

Então: o sistema registra a movimentação e exibe mensagem de sucesso

#Passou CT_92: Tentar Cadastrar Movimentação sem Preencher Campos Obrigatórios

Dado: que o usuário está no formulário de cadastro de movimentação de saída com campos obrigatórios vazios

Quando: o usuário clica no botão "Cadastrar"

Então: o sistema exibe mensagem de validação indicando campos obrigatórios não preenchidos e não prossegue com o cadastro

- ☐ (O ideal é a mensagem estar presente em todos os campos obrigatórios, preenchendo apenas os campos em que aparecem a mensagem quando você vai 'Cadastrar' novamente o sistema não mostra nenhum tipo de erro e não cadastra, a tela fica sem ação).

Título: *

O título da despesa é obrigatório!

Tipo de pagamento: *
  

Projeto:
 

Conta Debitada: *
 

Saldo da conta bancária:
Valor: R\$ 0,00

Descrição: *

Valor do pagamento: *

O valor da despesa é obrigatório!

Data: *
 
Selecione uma data!

Limpar **Cadastrar**

Título: *

Tipo de pagamento: *
  

Projeto:
 

Conta Debitada: *
 

Saldo da conta bancária:
Valor: R\$ 0,00

Descrição: *

Valor do pagamento: *

Data: *
 

Limpar **Cadastrar**

#Passou CT_93: Botão Voltar na Edição Retorna sem Salvar

Dado: que o usuário está no formulário de edição de uma movimentação com alterações não salvas

Quando: o usuário clica no botão "Voltar"

Então: o sistema retorna à página anterior sem salvar as alterações realizadas

Título: *

Pagto sistema conveniar TESTES

Tipo de pagamento: *

Pagamento de Fornecedores ADM Central

+

Projeto: *

Teste

Conta Debitada: *

Nome: TESTE NÚMERO 02 - Agência: 0912 - N°: 1231665

Saldo da conta bancária:

Valor: R\$ 152.000,31

Descrição: *

Pagto Sistema conveniar

Valor do pagamento: *

100.000,00

Data: *

06/05/2026

Voltar

Salvar

☐	Projeto	Teste	Pagto Sistema conveniar	—	- R\$ 100.000,00	R\$ 152.000,31
--------------------------	---------	-------	-------------------------	---	------------------	----------------

#Passou CT_94: Relatório de Movimentações — Pesquisa pelo Título

Dado: que o usuário está na tela de relatórios de movimentações com registros cadastrados

Quando: o usuário digita um título no campo de pesquisa

Então: o sistema filtra e exibe apenas as movimentações cujo título corresponde ao texto buscado

Pesquisar por nome:

PAGTO SISTEMA CONVENIAR

Download .xlsx

Digite o nome do arquivo...

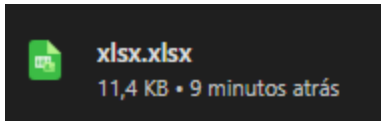
	Conta Bancária AZ	Projeto AZ	Descrição da despesa	Entrada	Saída	Sa
☐	Administrativa	Teste	Parcela administrativa referente a: Pagto Siste...	—	- R\$ 90.909,09	83
☐	Fundo de Reserva	Teste	Parcela fundo de reserva referente a: Pagto Sis...	—	- R\$ 9.090,90	14
☐	Projeto	Teste	Pagto Sistema conveniar	—	- R\$ 100.000,00	15

#Passou CT_95: Relatório de Movimentações — Download em .xlsx Sem Itens Selecionados

Dado: que o usuário está na tela de relatórios de movimentações com registros cadastrados e nenhum item selecionado

Quando: o usuário clica no botão de download .xlsx

Então: o sistema exporta todos os registros de movimentações no arquivo .xlsx



#Falhou CT_96: Relatório de Movimentações — Download em .xlsx com Itens Selecionados

Dado: que o usuário está na tela de relatórios de movimentações e selecionou 2 movimentações

Quando: o usuário clica no botão de download .xlsx

Então: o sistema exporta apenas os 2 registros selecionados no arquivo .xlsx

- ☐ (Mesmo selecionando as movimentações, é feita o download de todas as movimentações e não somente as que selecionei)

Pesquisar por nome:

PAGTO

Download .xlsx

Digite o nome do arquivo...



	Conta Bancária AZ	Projeto AZ	Descrição da despesa	Entrada	Saída	Saldo Total	Data AZ	Ações
☒	Administrativa	Teste	Parcela administrativa referente a: Pagto Siste...	—	- R\$ 90.909,09	R\$ 835.204,12	05/05/2026	  
☒	Fundo de Reserva	Teste	Parcela fundo de reserva referente a: Pagto Sis...	—	- R\$ 9.090,90	R\$ 141.783,32	05/05/2026	  
☐	Projeto	Teste	Pagto Sistema conveniar	—	- R\$ 100.000,00	R\$ 152.000,31	05/05/2026	  

#Passou CT_97: Visualizar Detalhes de uma Movimentação pelo Botão Visualizar

Dado: que o usuário está na tela de relatórios de movimentações

Quando: o usuário clica no botão "Visualizar" de uma movimentação

Então: o sistema exibe uma janela flutuante com todos os detalhes da movimentação e do projeto vinculado (descrição, saldo anterior, valor, saldo atual, data, conta bancária, nome e e-mail do coordenador)

Título:

Pagto sistema conveniar

Valor

R\$: 100.000,00

Data:

05/05/2026

Descrição:

Pagto Sistema conveniar

Projeto:**Título:**

Teste

Número:

21515161

Modalidade:

Desenvolvimento institucional

Conta bancaria:**Nome:**

TESTE NÚMERO 02

Agência:

0912

Número da Conta:

1231665

Titular:

ITALO

Saldo:

R\$: 152.000,31

Valor do instrumento:

R\$: 262.000,31

Valor da DOA:

R\$: 10.000,00

% da DOA:

3,82%

Valor do fundo de reserva:

R\$: 1.000,00

► DOA diluída:**Tipo no plano de trabalho:**

Tipo A

Status:

Em andamento

Data de início:

05/08/2024

Data de término:

30/07/2026

Nome do coordenador:

Bruna Castro Porto

E-mail do coordenador:

teste@mail.com

Descrição:

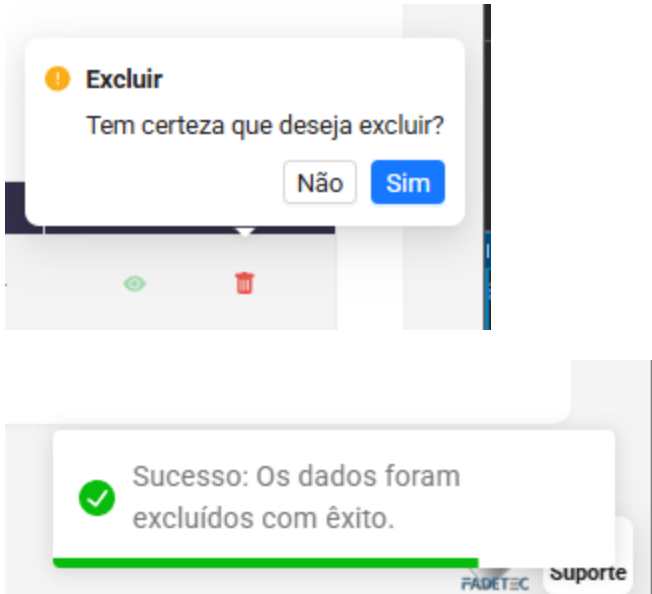
teste

Dado: que o usuário está na tela de relatórios de movimentações

Quando: o usuário clica no botão "Excluir" de uma movimentação existente

Então: o sistema exclui a movimentação e exibe feedback visual de sucesso

(O feedback de sucesso ou erro aparece no final esquerdo da página, fica confuso a leitura talvez seja melhor um modal centralizado com essa informação)



Investimentos — Cadastro (API — POST /investment)

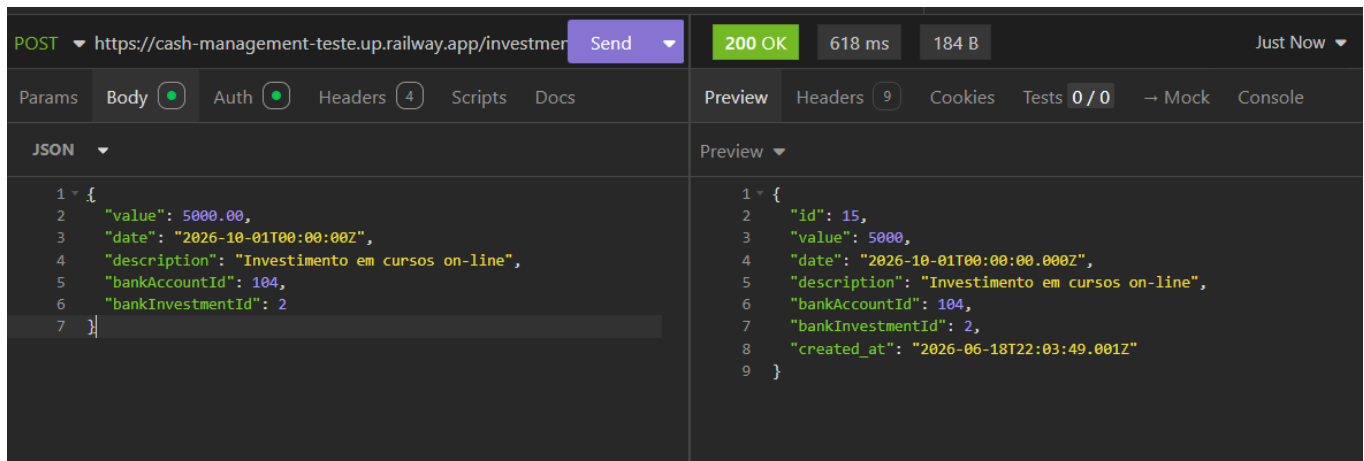
#Passou CT_99: Cadastro de Investimento com Todos os Campos Válidos

Dado: que o usuário está autenticado e existem uma conta de débito (não investimento) e uma conta de investimento cadastradas

Quando: é enviada uma requisição POST para `/investment` com o body:

```
{
  "value": "5000.00",
  "date": "2024-06-01T00:00:00Z",
  "description": "Investimento em cursos on-line",
  "bankAccountId": 1,
  "bankInvestmentId": 2
}
```

Então: o sistema retorna HTTP 200 com `"Inserido com sucesso!"` e o `id` do investimento criado



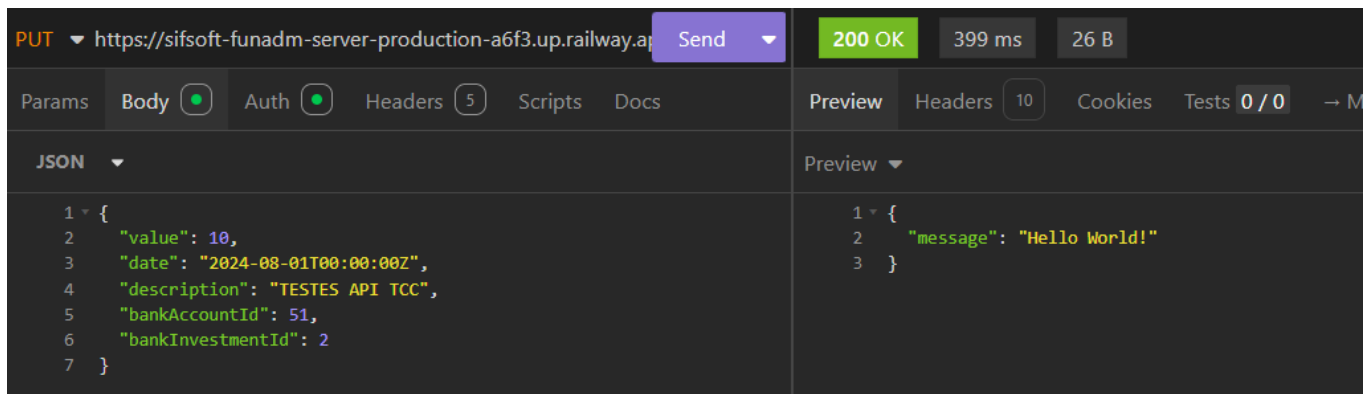
#Falhou CT_100: Cadastro de Investimento com Conta de Débito do Tipo "Investimento" (Regra de Negócio)

Dado: que o usuário está autenticado e a conta com `bankAccountId = 3` é do tipo "Investimento"

Quando: é enviada uma requisição POST para `/investment` com `bankAccountId = 3` (conta investimento como débito)

E: os demais campos estão válidos

Então: o sistema retorna erro bloqueando a operação, pois não é permitido debitar de uma conta do tipo Investimento



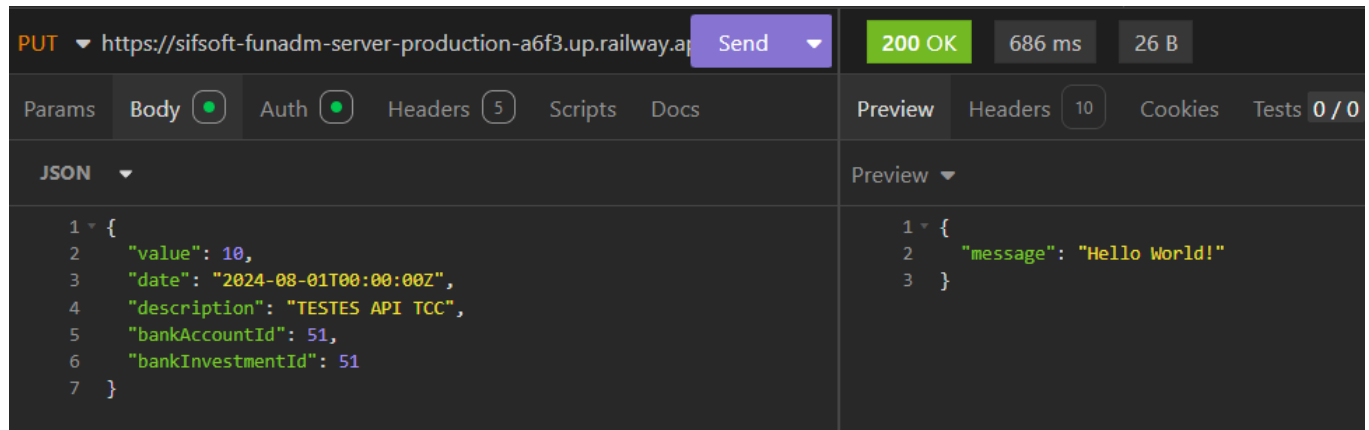
#Falhou CT_101: Cadastro de Investimento com Conta de Débito Igual à Conta de Investimento

Dado: que o usuário está autenticado

Quando: é enviada uma requisição POST para `/investment` com `bankAccountId = 2` e `bankInvestmentId = 2` (mesma conta)

E: os demais campos estão válidos

Então: o sistema retorna erro bloqueando a operação (conta de débito e conta de investimento não podem ser a mesma)



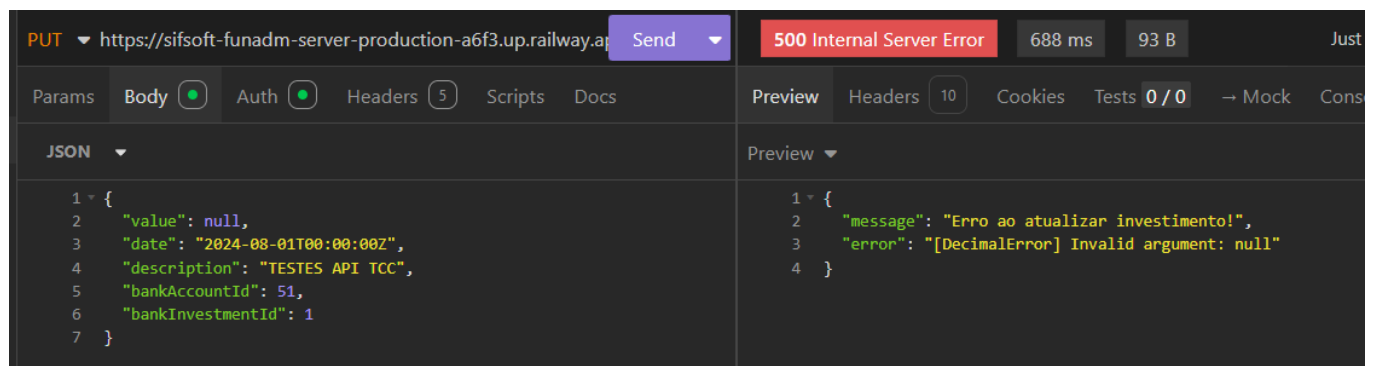
#Passou CT_102: Cadastro de Investimento com Valor Nulo

Dado: que o usuário está autenticado

Quando: é enviada uma requisição POST para `/investment` com `value = null`

E: os demais campos estão válidos

Então: o sistema retorna HTTP 500 — Internal Server Error



#Falhou CT_103: Cadastro de Investimento com Valor Negativo

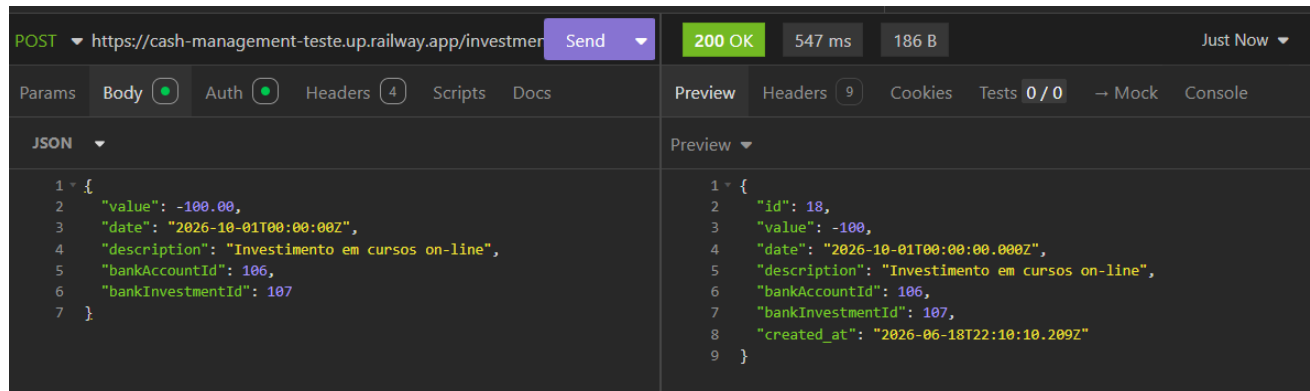
Dado: que o usuário está autenticado

Quando: é enviada uma requisição POST para `/investment` com `value = "-1000.00"`

E: os demais campos estão válidos

Então: o sistema retorna erro de validação ou HTTP 500

☐ (Pela API não realiza o controle desse valor negativo)



```
POST https://cash-management-teste.up.railway.app/investmer 200 OK 547 ms 186 B Just Now
Params Body Auth Headers 4 Scripts Docs Preview Headers 9 Cookies Tests 0/0 → Mock Console
JSON Preview
1 {
2   "value": -100.00,
3   "date": "2026-10-01T00:00:00Z",
4   "description": "Investimento em cursos on-line",
5   "bankAccountId": 106,
6   "bankInvestmentId": 107
7 }
```

```
1 {
2   "id": 18,
3   "value": -100,
4   "date": "2026-10-01T00:00:00.000Z",
5   "description": "Investimento em cursos on-line",
6   "bankAccountId": 106,
7   "bankInvestmentId": 107,
8   "created_at": "2026-06-18T22:10:10.209Z"
9 }
```

#Passou CT_104: Cadastro de Investimento com Valor Grande (NUMERIC 16,2)

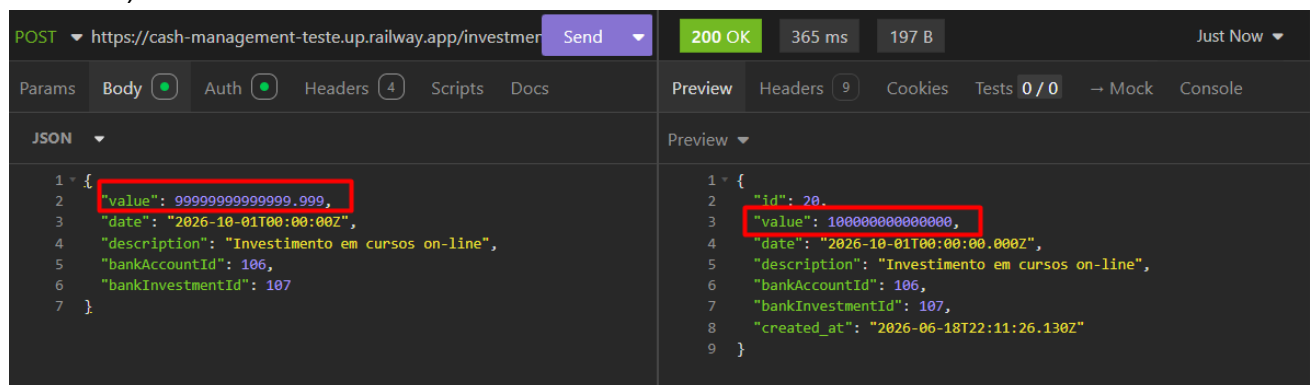
Dado: que o usuário está autenticado

Quando: é enviada uma requisição POST para `/investment` com `value = "9999999999999999.99"`

E: os demais campos estão válidos

Então: o sistema retorna HTTP 200 com `"Inserido com sucesso!"`

☐ (Aceitou mais caracteres que o NUMERIC(16,2)) e quando isso ocorreu converteu o número.)



```
POST https://cash-management-teste.up.railway.app/investmer 200 OK 365 ms 197 B Just Now
Params Body Auth Headers 4 Scripts Docs Preview Headers 9 Cookies Tests 0/0 → Mock Console
JSON Preview
1 {
2   "value": 9999999999999999.999,
3   "date": "2026-10-01T00:00:00Z",
4   "description": "Investimento em cursos on-line",
5   "bankAccountId": 106,
6   "bankInvestmentId": 107
7 }
```

```
1 {
2   "id": 20,
3   "value": 10000000000000000,
4   "date": "2026-10-01T00:00:00.000Z",
5   "description": "Investimento em cursos on-line",
6   "bankAccountId": 106,
7   "bankInvestmentId": 107,
8   "created_at": "2026-06-18T22:11:26.130Z"
9 }
```

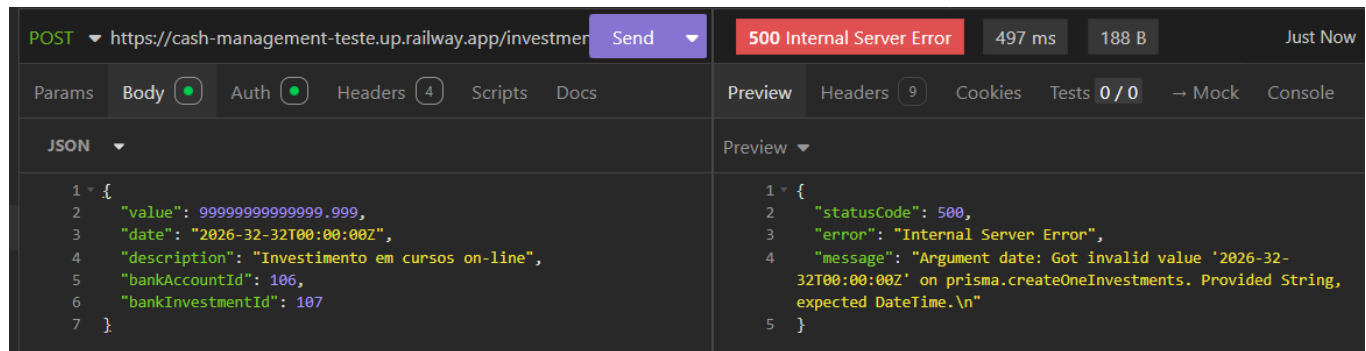
#Passou CT_105: Cadastro de Investimento com Data em Formato Inválido

Dado: que o usuário está autenticado

Quando: é enviada uma requisição POST para `/investment` com `date = "data-invalida"`

E: os demais campos estão válidos

Então: o sistema retorna HTTP 500 — Internal Server Error



```
POST https://cash-management-teste.up.railway.app/investment Send 500 Internal Server Error 497 ms 188 B Just Now
```

Params Body Auth Headers 4 Scripts Docs Preview Headers 9 Cookies Tests 0/0 → Mock Console

JSON

```
1 {
2   "value": 999999999999.999,
3   "date": "2026-32-32T00:00:00Z",
4   "description": "Investimento em cursos on-line",
5   "bankAccountId": 106,
6   "bankInvestmentId": 107
7 }
```

Preview

```
1 {
2   "statusCode": 500,
3   "error": "Internal Server Error",
4   "message": "Argument date: Got invalid value '2026-32-32T00:00:00Z' on prisma.createOneInvestments. Provided String, expected DateTime.\n"
5 }
```

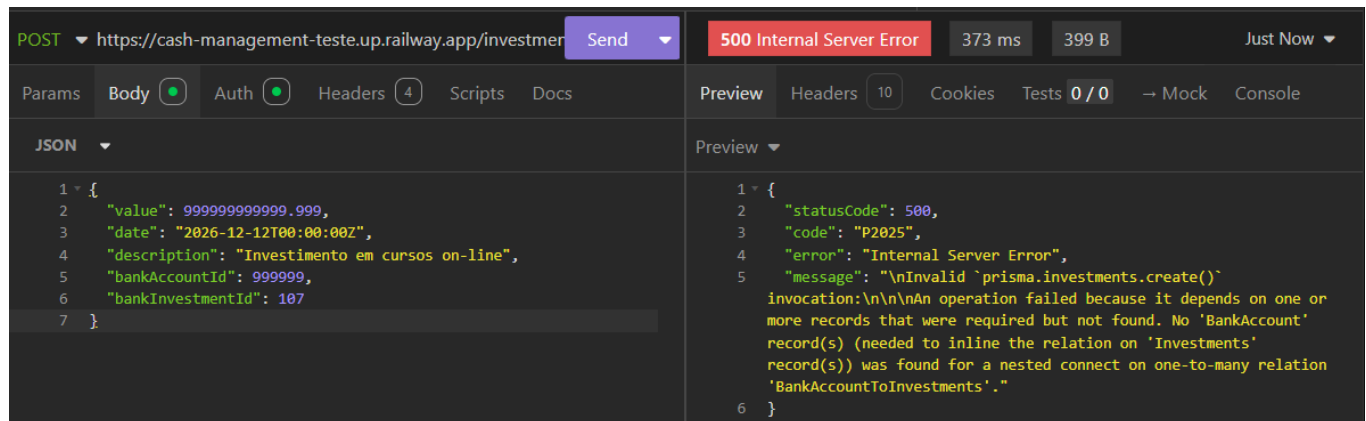
#Passou CT_106: Cadastro de Investimento com bankAccountId Inexistente

Dado: que o usuário está autenticado

Quando: é enviada uma requisição POST para `/investment` com `bankAccountId = 9999`

E: os demais campos estão válidos

Então: o sistema retorna HTTP 500 — Internal Server Error por violação de chave estrangeira



```
POST https://cash-management-teste.up.railway.app/investment Send 500 Internal Server Error 373 ms 399 B Just Now
```

Params Body Auth Headers 4 Scripts Docs Preview Headers 10 Cookies Tests 0/0 → Mock Console

JSON

```
1 {
2   "value": 999999999999.999,
3   "date": "2026-12-12T00:00:00Z",
4   "description": "Investimento em cursos on-line",
5   "bankAccountId": 999999,
6   "bankInvestmentId": 107
7 }
```

Preview

```
1 {
2   "statusCode": 500,
3   "code": "P2025",
4   "error": "Internal Server Error",
5   "message": "\nInvalid `prisma.investments.create()`\ninvocation:\n\nAn operation failed because it depends on one or\nmore records that were required but not found. No 'BankAccount'\nrecord(s) (needed to inline the relation on 'Investments'\nrecord(s)) was found for a nested connect on one-to-many relation\n'BankAccountToInvestments'.
6 }
```

#Passou CT_107: Cadastro de Investimento sem Token de Autenticação

Dado: que o usuário não está autenticado

Quando: é enviada uma requisição POST para `/investment` com body válido e sem o header `Authorization`

Então: o sistema retorna HTTP 401 — Unauthorized

```
POST https://cash-management-teste.up.railway.app/investmen Send 401 Unauthorized 460 ms 143 B Just Now
Params Body Auth Headers 4 Scripts Docs Preview Headers 9 Cookies Tests 0/0 → Mock Console
JSON Preview
1 {
2   "value": 9999999999.999,
3   "date": "2026-12-12T00:00:00Z",
4   "description": "Investimento em cursos on-line",
5   "bankAccountId": 132,
6   "bankInvestmentId": 107
7 }
```

Investimentos — Listagem e Busca (GET /investment e GET /investment/{id})

#Passou CT_108: Listar Todos os Investimentos com Token Válido

Dado: que o usuário está autenticado

Quando: é enviada uma requisição GET para /investment

Então: o sistema retorna HTTP 200 com a lista de investimentos cadastrados

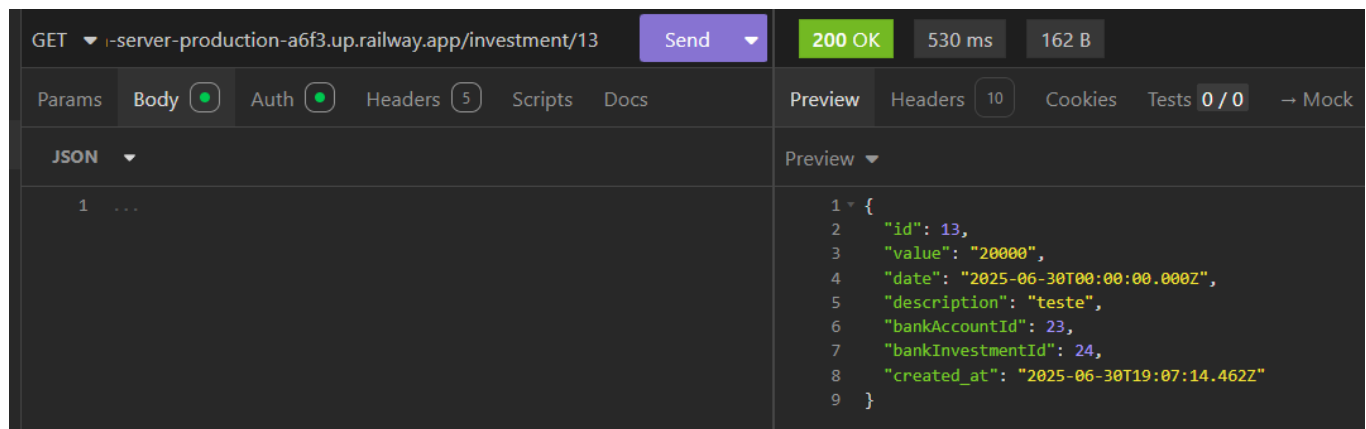
```
GET https://sifsoft-funadm-server-production-a6f3.up.railway.ap Send 200 OK 1.21 s 5.8 KB Just N
Params Body Auth Headers 5 Scripts Docs Preview Headers 11 Cookies Tests 0/0 → Mock Conso
JSON Preview
1 |...
87   "description": "teste",
88   "bankAccountId": 23,
89   "bankInvestmentId": 16,
90   "created_at": "2025-06-30T19:06:54.143Z"
91 },
92 {
93   "id": 13,
94   "value": "20000",
95   "date": "2025-06-30T00:00:00.000Z",
96   "description": "teste",
97   "bankAccountId": 23,
98   "bankInvestmentId": 24,
99   "created_at": "2025-06-30T19:07:14.462Z"
100 },
101 {
102   "id": 14,
103   "value": "21000",
104   "date": "2025-06-30T00:00:00.000Z",
105   "description": "teste",
106   "bankAccountId": 12,
107   "bankInvestmentId": 24,
108   "created_at": "2025-06-30T19:07:46.428Z"
109 },
```

#Passou CT_109: Buscar Investimento por ID Existente

Dado: que existe um investimento com id = 13 e o usuário está autenticado

Quando: é enviada uma requisição GET para `/investment/13`

Então: o sistema retorna HTTP 200 com os dados completos do investimento

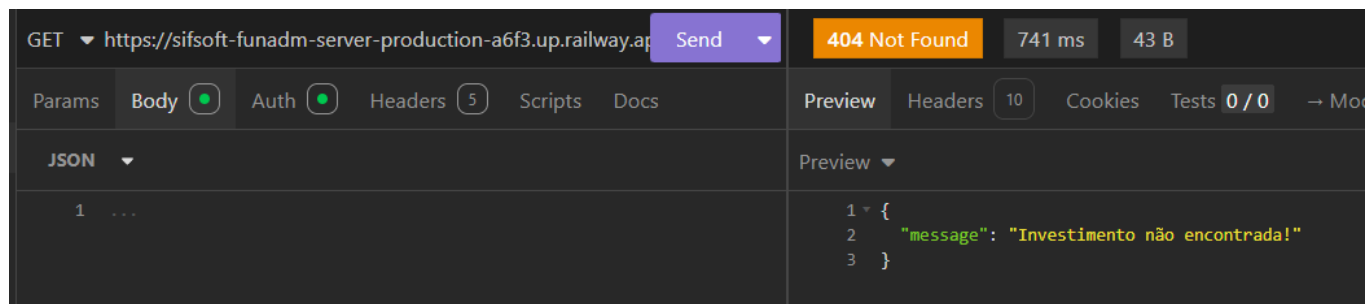


#Passou CT_110: Buscar Investimento por ID Inexistente

Dado: que o usuário está autenticado

Quando: é enviada uma requisição GET para `/investment/9999`

Então: o sistema retorna HTTP 404 — Not Found



Investimentos — Edição (PUT `/investment/{id}`)

#Passou CT_111: Editar Valor e Descrição do Investimento com Dados Válidos

Dado: que existe um investimento com `id = 12` e o usuário está autenticado

Quando: é enviada uma requisição PUT para `/investment/12` com o body:

```
{
  "value": "8000.00",
  "date": "2024-08-01T00:00:00Z",
  "description": "Atualização do investimento",
}
```

```
"bankAccountId": 1,  
"bankInvestmentId": 2  
}
```

Então: o sistema retorna HTTP 200 e os dados do investimento são atualizados

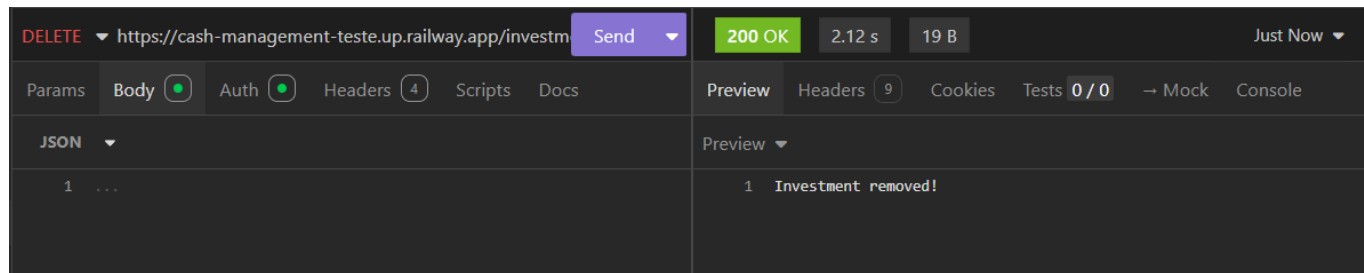
Investimentos — Exclusão (DELETE /investment/{id})

#Passou CT_112: Excluir Investimento Existente

Dado: que existe um investimento com `id = 10` e o usuário está autenticado

Quando: é enviada uma requisição DELETE para `/investment/10`

Então: o sistema retorna HTTP 200 com mensagem de sucesso



#Passou CT_113: Excluir Investimento com ID Inexistente

Dado: que o usuário está autenticado

Quando: é enviada uma requisição DELETE para `/investment/9999`

Então: o sistema retorna HTTP 500 — Internal Server Error ou 404

Investimentos — Tela (Frontend)

#Passou CT_114: Campo "Conta para Débito" Não Lista Contas do Tipo Investimento

Dado: que o usuário está no formulário de cadastro de investimento

Quando: o usuário acessa o campo "Selecione a conta para o débito"

Então: contas do tipo "Investimento" não aparecem como opção de débito

Selecione a conta para o débito: *

Selecione conta bancária...

+ Administrativa

+ Fundo de reserva

+ Projetos

#Passou CT_115: Verificar Exibição do Saldo da Conta Debitada ao Selecionar a Conta

Dado: que o usuário está no formulário de cadastro de investimento

Quando: o usuário seleciona uma conta no campo "Selecione a conta para o débito"

Então: o sistema exibe o saldo atual da conta selecionada no campo "Saldo da conta bancária debitada"

Selecione a conta para o débito: *

Nome: testeq - Agência: 123 N°: 456

Saldo da conta bancária debitada:

Valor:	R\$ 1.108.299,38
--------	------------------

#Passou CT_116: Verificar Máscara do Campo "Data" no Formulário de Investimento (dd/mm/aaaa)

Dado: que o usuário está no formulário de cadastro de investimento

Quando: o usuário digita uma data no campo "Data"

Então: o sistema aplica a máscara dd/mm/aaaa automaticamente

Data: *

10/10/2020

#Passou CT_117: Botão Limpar Esvazia Todos os Campos do Formulário de Investimento

Dado: que o usuário preencheu os campos do formulário de cadastro de investimento

Quando: o usuário clica no botão "Limpar"

Então: todos os campos são redefinidos ao estado inicial



Seleção de conta para o débito: *

Seleção de conta bancária... ▼

Saldo da conta bancária debitada:

Valor: R\$ 0,00

Selecionar conta de investimento para depósito: *

Seleção de conta bancária... ▼

Descrição: *

Digite a descrição...

Valor: *

Valor: Digite o valor...

Data: *

Data: dd/mm/aaaa

Limpar

Cadastrar

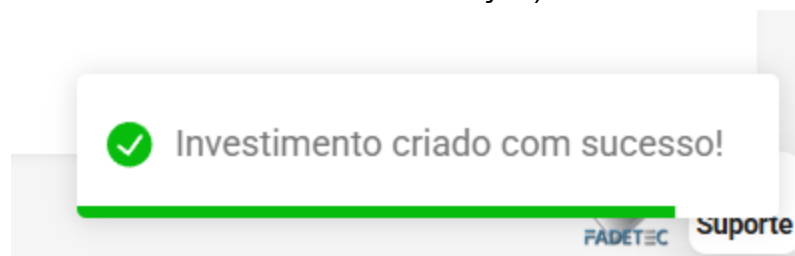
#Passou CT_118: Botão Cadastrar Registra Investimento com Campos Válidos

Dado: que o usuário preencheu corretamente todos os campos do formulário de investimento

Quando: o usuário clica no botão "Cadastrar"

Então: o sistema registra o investimento e exibe mensagem de sucesso

- ☐ (A mensagem de sucesso aparece no fim da página, o interessante seria algo como uma modal central com essa informação)



#Passou CT_119: Relatório de Investimentos — Pesquisa pela Descrição

Dado: que o usuário está na tela de relatórios de investimentos com registros cadastrados

Quando: o usuário digita uma descrição no campo de pesquisa

Então: o sistema filtra e exibe apenas os investimentos cuja descrição corresponde ao texto buscado

The screenshot shows the 'Investimentos' (Investments) report interface. At the top, there are tabs for 'Projetos', 'Movimentações', 'Investimentos' (selected), 'DOA Diluída.', 'Extrato', and 'TimeLine'. Below the tabs, there is a search bar labeled 'Pesquisar Investimento por descrição:' with the text 'TESTE' entered. To the right of the search bar is a 'Download .xlsx' button. Below the search bar, there is a table with the following columns: 'Nº', 'Descrição', 'Valor do investimento', 'Data', 'Conta debitada', 'Conta investida', and 'Ações'. The table contains two rows of data.

Nº	Descrição	Valor do investimento	Data	Conta debitada	Conta investida	Ações
7	teste 2	500,00	01/05/2025	Nome: Sicoob - 3144-5 - 65735-2	Nome: Sicoob - 3144-5 - 45879-1	
47	Teste - Teste	1,00	05/06/2020	Nome: teste - 123456 - 060 123456147 8	Nome: Sicoob - 3144-5 - 45879-1	

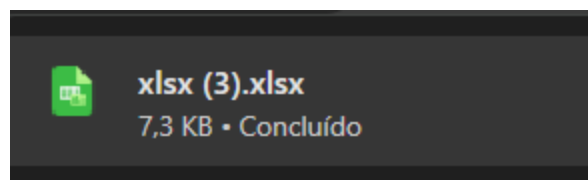
#Passou CT_120: Relatório de Investimentos — Download em .xlsx Sem Itens

Selecionados

Dado: que o usuário está na tela de relatórios de investimentos com registros cadastrados e nenhum item selecionado

Quando: o usuário clica no botão de download .xlsx

Então: o sistema exporta todos os registros de investimentos no arquivo .xlsx



#Falhou CT_121: Relatório de Investimentos — Download em .xlsx com Itens

Selecionados

Dado: que o usuário está na tela de relatórios de investimentos e selecionou 1 investimento

Quando: o usuário clica no botão de download `.xlsx`

Então: o sistema exporta apenas o registro selecionado no arquivo `.xlsx`

☐ (Mesmo selecionando, o arquivo baixado vem o geral)

Pesquisar Investimento por descrição:

Pesquisar investimento...

Q

Download .xlsx

Digite o nome do arquivo...



	N° AZ	Descrição AZ	Valor do investimento AZ	Data AZ	Conta debitada	Conta investida	Ações
☒	2	Aplicação	10.000,00	01/04/2025	Nome: Sicoob - 3144-5 - 64144-8	Nome: Sicoob - 3144-5 - 546897-8	  

#Passou CT_122: Visualizar Detalhes de um Investimento pelo Botão Visualizar

Dado: que o usuário está na tela de relatórios de investimentos

Quando: o usuário clica no botão "Visualizar" de um investimento

Então: o sistema exibe uma janela flutuante com todos os detalhes: descrição, valor, data, conta bancária debitada (nome, agência, número, titular, saldo, tipo) e conta bancária investida (nome, agência, número, titular, saldo, tipo)

Descrição:

teste 2

Valor

R\$: 500,00

Data:

01/05/2025

Conta bancária debitada:
-----**Nome do banco:**

Sicoob

Conta bancária:

3144-5

Número da conta:

65735-2

Titular:

Fadotec Fundo de Reserva

Saldo:

R\$: 141.783,32

Tipo de conta:

Fundo de Reserva

-----**Conta bancária investida:**
-----**Nome do banco:**

Sicoob

Conta bancária:

3144-5

Número da conta:

45879-1

Titular:

Investimento 2

Saldo:

R\$: 119.901,00

Tipo de conta:

Investimento

[#Passou](#) **CT_123: Excluir Investimento pelo Botão Excluir na Tela de Relatórios**

Dado: que o usuário está na tela de relatórios de investimentos

Quando: o usuário seleciona um investimento e clica no botão "Excluir"

Então: o sistema exclui o investimento e exibe feedback visual de sucesso

- ☐ (O modal fica perto da linha esquerda, a visualização fica defasada, ideal seria centralização)

